

Ingeniería de Sistemas



Hub TuCasaConstrucciones – ERP de gestión constructora
Documento de especificación del diseño e implementación de la Base de Datos

Autores
Cristian Danilo Areniz Coronel - 0192337
Santiago Castilla Coronel - 0192413
Luis Felipe Carrascal Ortiz - 0192391

Versión 1.2

Control de cambios

Fecha	Versión	Descripción	Autor
23/08/2026	**1.0.1**	**Se agrego los requisitos de información y las reglas del negocio**	**Cristian Danilo Areniz**
23/08/2026	**1.0.2**	**Se agrego la introducción y el negocio y mision**	**Santiago Castilla**
23/08/2026	**1.0.3**	**Se agrego los supuestos**	**Luis Felipe Carrascal**
07/11/2026	1.1.0	Se completaron los modelos conceptual, lógico y físico, el diseño, la creación (DDL/DML, C4, repositorio), la transición (plan UAT), la producción, conclusiones, recomendaciones, glosario y bibliografía	Cristian Danilo Areniz
09/11/2026	1.2.0	Se ajustó todo el documento a la base de datos real desplegada en Supabase (copia del proyecto base_de_datos): 29 tablas, 5 triggers y 5 funciones. Se reescribieron requisitos, reglas, supuestos, diseño, DDL/DML, C4, pruebas UAT, producción, conclusiones y recomendaciones, y se agregó el Anexo 1 (evidencias). Se completaron los datos de la copia (carga complementaria), se entregó el respaldo de datos como script DML, se agregó la matriz de trazabilidad y se publicó el repositorio de la entrega.	Santiago Castilla

Tabla de contenido.

Portada
1. Introducción
2. Estrategia y Análisis
- **2.1 Negocio y misión**
- **2.2 Requisitos de información**
- **2.3 Reglas del negocio**
- **2.4 Supuestos**
- **2.5 Modelo conceptual (Diagrama de ER)**
- **2.6 Modelo lógico (Diagrama de ER)**

3. Diseño
- **3.1 Especificaciones de diseño**
- **3.2 Transformación del modelo lógico a Diagrama de tablas o grafico de instancias de tablas.**

4. Creación
- **4.1 Modelo físico.**
- **4.2 Script DDL y DML**
- **4.3 Modelo C4 (Nivel 1: Diagrama contexto sistema, Nivel 2: Diagrama contenedor)**
- **4.4 Desarrollo web**
- **4.5 Repositorio Github**

5. Transición
- **5.1 Pruebas de aceptación**

6. Producción
- **6.1 Desplegar el modelo físico en la nube**
- **6.2 Colocar en funcionamiento el sistema**

7. Conclusión
8. Recomendaciones
9. Glosario
10. Bibliografía
Anexo 1. Instrumento de recolección de evidencias

Portada

1. Introducción

En la actualidad, las familias se enfrentan a la constante necesidad de encontrar espacios habitables que no solo cubran sus requerimientos básicos de resguardo, sino que verdaderamente ofrezcan calidad, confort, seguridad y un entorno propicio para el bienestar integral y el desarrollo familiar. A menudo, el mercado inmobiliario convencional ofrece edificaciones frías o impersonales que descuidan el valor emocional y la durabilidad a largo plazo de un hogar. Como solución a esta necesidad, TuCasaConstrucciones se establece como una empresa especializada en el desarrollo y construcción de viviendas orientada a materializar las aspiraciones de las familias. La compañía propone un enfoque humano e innovador en la arquitectura residencial, diseñando y construyendo espacios de alta calidad enfocados en fomentar estilos de vida saludables, forjar recuerdos inolvidables y garantizar un legado duradero para sus clientes.

Este documento especifica el diseño e implementación de la base de datos del Hub TuCasaConstrucciones tal como quedó desplegada en Supabase (PostgreSQL 17): 29 tablas organizadas en siete módulos (terceros y acceso, proyectos, ventas, compromisos y contratos, tesorería, obras y control, y catálogos), con restricciones de integridad, columnas calculadas, cinco funciones y cinco triggers, y datos sintéticos de prueba generados con IA (500 proyectos, 1000 actividades de proyecto, 500 compromisos, 500 movimientos de tesorería y 200 ventas, entre otros).

2. Estrategia y Análisis

2.1 Negocio y misión

- **¿Cuáles son los objetivos del negocio?**

Centrarnos en liderar el sector de la construcción de viviendas a través de la innovación constante, la responsabilidad ambiental y el bienestar integral. Para lograrlo, buscamos superar las expectativas de nuestros clientes creando hogares seguros, cómodos y de alta calidad que no solo satisfagan necesidades habitacionales, sino que promuevan estilos de vida saludables, fomenten recuerdos inolvidables y dejen un legado duradero.

- **¿Qué tipo de negocio es ó desea ser?**

TuCasaConstrucciones es una empresa dedicada a la construcción y desarrollo de viviendas familiares, con el firme propósito de consolidarse como el líder del sector gracias a su enfoque en la innovación continua y la responsabilidad ambiental.

- **¿Cómo se perciban los servicios?**

Los servicios se perciben como una propuesta integral y profundamente humana, donde la calidad, la seguridad y el confort se combinan para crear "hogares de sueños" diseñados para garantizar el bienestar, la felicidad y un legado duradero a las familias.

2.2 Requisitos de información

Evaluación de datos históricos:

- **¿Es necesario hacer una auditoría?**

Sí. Es necesario llevar un registro exacto de las transacciones financieras: cada movimiento de tesorería (treasury_movements) guarda su fecha, monto, tipo, el perfil que lo creó (created_by) y el momento de creación (created_at). Las ventas y los compromisos también registran created_by y created_at. Los cambios de estado de las ventas y de los inmuebles (por ejemplo de available a sold) se sincronizan automáticamente mediante un trigger de la base de datos.

- **¿Puede cambiar los valores de atributos a lo largo del tiempo?**

Sí. El precio de lista de un inmueble (properties.total_price) puede variar, por eso el valor realmente pactado se guarda aparte en cada venta (sales.total_sale_value) y no cambia si el precio de lista se actualiza. El avance de los proyectos (projects.progress) y de las actividades de obra (work_progress) se actualiza periódicamente, y las tarifas de retención se versionan por vigencia (retention_type_rates.effective_from y effective_to) para que un cambio de ley no altere facturas anteriores.

- **¿Pueden cambiar las relaciones a lo largo del tiempo?**

Sí. Un tercero registrado como cliente (users.user_type = client) cambia su relación con la empresa cuando se registra una venta (sales) sobre un inmueble: la venta pasa por los estados active, completed o canceled y, según ese estado, el inmueble cambia entre available y sold. Un mismo cliente puede comprar varios inmuebles.

- **¿Necesita producir informes sobre datos antiguos?**

Sí. La gerencia requiere analizar el comportamiento de los proyectos finalizados (projects.status = completed) mediante consultas sobre ventas, presupuesto (project_activities.budgeted_cost)

y costos reales (work_progress.actual_cost) para proyectar la viabilidad de futuras construcciones.

- **¿Necesita conservar versiones antiguas de los datos? Si es así, ¿durante cuánto tiempo?**

Sí. Dando cumplimiento a la normatividad colombiana, los documentos de carácter contable y legal (promesas de compraventa, registros de entrega) deben conservarse por un mínimo de 10 años. Asimismo, la información estructural y técnica de los proyectos debe mantenerse archivada para responder a las garantías legales contempladas en el Estatuto del Consumidor.

Requerimientos de Información Identificados:

ID	Requerimiento
1	Se requiere información de cada tercero (users): tipo (cliente, proveedor, contratista o empleado), tipo y número de documento (CC, NIT, CE o PASSPORT), nombre, teléfono, correo electrónico, soportes del RUT y de la cédula (URL) y sus cuentas bancarias (banco, tipo de cuenta, número y cuenta principal).
2	Se requiere gestionar los perfiles de acceso (profiles) vinculados a la autenticación, con un rol (superadmin, admin, accountant, foreman o assistant), y vincular opcionalmente un perfil a un tercero.
3	Se necesita registrar cada proyecto inmobiliario con nombre único, ubicación, estado (active, completed u on_hold), fechas de inicio y fin y porcentaje de avance (0 a 100), dividido en bloques (viviendas o zonas comunes) y con sus inmuebles: código, tipo, área en m², precio y estado comercial (available, reserved o sold).
4	Se debe gestionar la venta de los inmuebles: cliente, inmueble, fecha, valor de separación, valor total, cuota inicial, crédito (valor total menos cuota inicial), estado de la venta (active, completed o canceled) y carpeta de soportes.
5	El sistema debe almacenar el plan de pagos de cada venta como cuotas numeradas, con fecha de

ID	Requerimiento
	vencimiento, valor esperado y tipo (cuota inicial, crédito hipotecario, subsidio o extra), y registrar cómo se aplican los pagos recibidos a cada cuota.
6	Se requiere registrar los compromisos de pago (facturas, cuentas de cobro y liquidaciones de contrato): acreedor, proyecto y contrato asociados, formato, número, fecha, valor bruto, IVA, retención, anticipo, total y neto a pagar, con sus ítems (descripción, cantidad, precio unitario, capítulo y actividad).
7	Se necesita administrar los tipos de retención y sus tarifas porcentuales con fecha de vigencia desde y hasta, para aplicarlas a compromisos e ítems.
8	Se requiere registrar los contratos con contratistas (proyecto, capítulo, objeto, valor estimado y estado: active, suspended, pending_closure o closed) y sus actas de avance (valor ejecutado, fecha y soporte).
9	El sistema debe llevar la tesorería: cuentas de la empresa (banco, caja menor, efectivo o inversión) y movimientos con tipo y subtipo, fecha, monto, tercero, proyecto, compromiso, venta o contrato asociado, medio de pago, cuentas de origen y destino y soporte; y aplicar los pagos a compromisos y a cuotas de venta.
10	Se requiere controlar la obra: presupuesto por actividad (cantidad, precio y costo calculado, fechas planeadas y reales) con un catálogo de actividades por proyecto, avance de obra (cantidad ejecutada y precio real) y rendimiento del personal (cuadrilla por tipo de mano de obra, contratistas, días trabajados y tasa de rendimiento).
11	Se requieren catálogos maestros con nombres o códigos únicos: bancos, medios de pago, capítulos, unidades de medida, tipos de mano de obra y tipos de retención.
12	Se debe conservar la fecha de creación y de modificación de los registros principales y el perfil que los creó, para efectos de auditoría y de los informes históricos.

2.3 Reglas del negocio

ID	Regla	Tipo
1	Todo tercero (users) tiene un tipo (client, provider, contractor o employee) y un tipo de documento (CC, NIT, CE o PASSPORT) con su número; un perfil de acceso se asocia	E

ID	Regla	Tipo
	como máximo a un tercero.	
2	Un proyecto se identifica por un nombre único, tiene un estado (active, completed u on_hold) y un avance entre 0 y 100.	E
3	Un proyecto se divide en bloques, y cada bloque es de tipo housing (viviendas) o common_areas (zonas comunes).	E
4	Un inmueble pertenece a un bloque del mismo proyecto, tiene área mayor que 0, precio mayor o igual a 0 y un estado (available, reserved o sold).	E
5	Un inmueble solo puede asignarse a un bloque de tipo housing; la base de datos lo verifica con un trigger.	P
6	Al registrar una venta activa o completada, el inmueble pasa automáticamente a sold; si la venta se cancela, vuelve a available.	P
7	Una venta relaciona un cliente con un inmueble; un mismo cliente puede comprar más de un inmueble. El valor del crédito se calcula como valor total menos cuota inicial.	E
8	El plan de pagos de una venta se compone de cuotas numeradas con fecha de vencimiento, valor mayor que 0 y un tipo (down_payment_installment, mortgage_credit, subsidy o extra).	E
9	Todo ingreso por pago de venta (income / sale_payment) se aplica automáticamente a las cuotas más antiguas primero (FIFO); si sobra dinero, queda como crédito a favor del cliente.	P
10	Un compromiso es una factura, una cuenta de cobro o una liquidación de contrato, tiene un acreedor (tercero) y puede asociarse a un proyecto y a un contrato. Sus valores bruto, IVA, retención y anticipo son mayores o iguales a 0.	E
11	El total del compromiso (bruto + IVA) y el neto a pagar (total - retención - anticipo) los calcula la base de datos y no se digitan.	E
12	Un compromiso se detalla en ítems con cantidad mayor que 0 y precio unitario mayor o igual a 0; el total de cada ítem se calcula.	E
13	Las retenciones se clasifican por tipo y tienen tarifas porcentuales (0 a 100 %) con fecha de vigencia desde y hasta.	E

ID	Regla	Tipo
14	Un contrato asocia un contratista con un proyecto, tiene un valor estimado mayor que 0 y un estado (active, suspended, pending_closure o closed); su ejecución se certifica con actas de avance.	E
15	Solo un tercero de tipo contractor puede ser el contratista de un contrato; la base de datos lo verifica con un trigger.	P
16	Todo movimiento de tesorería tiene monto mayor que 0, un tipo (income, expense, transfer o investment), un subtipo (sale_payment, sale_refund, contract_advance, commitment_payment, petty_cash_funding, petty_cash_expense, internal_transfer, cdt o loan) y el perfil que lo creó.	E
17	Las cuentas de la empresa tienen nombre único y una clase (bank, petty_cash, cash o investment); los movimientos pueden indicar cuenta de origen y de destino.	E
18	Cada actividad del presupuesto pertenece a un proyecto y a un bloque de ese mismo proyecto, con cantidad mayor que 0; su costo presupuestado se calcula (cantidad x precio). El avance de obra y el rendimiento del personal se registran contra la actividad.	E
19	Cada vez que se modifica un proyecto, su campo updated_at se actualiza automáticamente (trigger).	P
20	Los catálogos (bancos, medios de pago, capítulos, unidades, tipos de mano de obra y tipos de retención) tienen nombre o código únicos.	E

E: Estructural. Indican los tipos de información que se van a almacenar (atributos) y como se interrelacionan los elementos de información (relaciones). Se puede representar en el diagrama ERD.

P: Procedimiento. Representa pasos, procesos ó flujos de trabajo de un negocio. Generalmente no se puede representar en el diagrama ERD. "Implantar mediante programación" significa que, una vez creada la base de datos, un programador escribirá código que aplicaría una regla de negocio.

2.4 Supuestos

ID	Supuesto

ID	Supuesto
1	Los valores monetarios se manejan en pesos colombianos (COP) con tipo NUMERIC, sin símbolo de moneda.
2	Cada usuario autenticado de Supabase tiene un perfil (profiles) con un rol (superadmin, admin, accountant, foreman o assistant); los permisos por rol se aplican en la capa de aplicación, porque la base entregada no activa RLS.
3	Los saldos y estados derivados (valor pagado y saldo de una factura, saldo de una cuenta, estado de una cuota) no se almacenan: se calculan con consultas sobre treasury_movements, commitment_applications y payment_applications.
4	Los datos de contacto del tercero (teléfono, correo) no son llave de identificación; la identificación es el tipo y número de documento, y se asume que la aplicación evita registrar dos terceros con el mismo documento (la base no tiene esa restricción única).
5	Un inmueble tiene a lo sumo una venta vigente (active o completed); se asume que la aplicación lo valida, y se recomienda reforzarlo con un índice único parcial en la base.
6	Los documentos de soporte (RUT, cédula, facturas, carpeta de la venta) se guardan como URL a un almacenamiento externo, no dentro de la base de datos.
7	Todas las llaves primarias son UUID generados por la base (gen_random_uuid()), excepto profiles, cuyo id es el del usuario de autenticación, y las tablas intermedias con llave compuesta.
8	Los datos poblados son sintéticos, generados con IA y validados con consultas SQL; no corresponden a clientes, proveedores ni transacciones reales.
9	Un proyecto puede tener varias tipologías de inmueble, que se describen con properties.property_type, area_sqm y total_price (no existe una tabla de tipologías).

2.5 Modelo conceptual (Diagrama de ER)

- El modelo conceptual representa las 25 entidades del negocio y sus relaciones, sin atributos ni tipos de dato. Se organiza por los módulos del ERP: Terceros y acceso (azul), Proyectos (verde), Compromisos y Contratos (amarillo), Tesorería (rosado), Obras y Control (violeta), Ventas (turquesa) y Catálogos maestros (gris). Al pasar al modelo físico, las relaciones muchos a muchos se resuelven con cuatro tablas intermedias (commitment_applications, payment_applications, labor_performance_contractors y labor_performance_crew), por lo que las 25 entidades se convierten en 29 tablas.
- El modelo se refinó con la empresa durante el levantamiento de entidades y la validación de reglas de negocio, lo que llevó a incorporar entidades de apoyo como Bloque (que permite modelar zonas comunes), Catálogo de actividades y Tarifa de retención (versionada por vigencia). La nomenclatura usa singular en el diagrama conceptual y nombres de tabla en inglés y plural en el modelo físico.

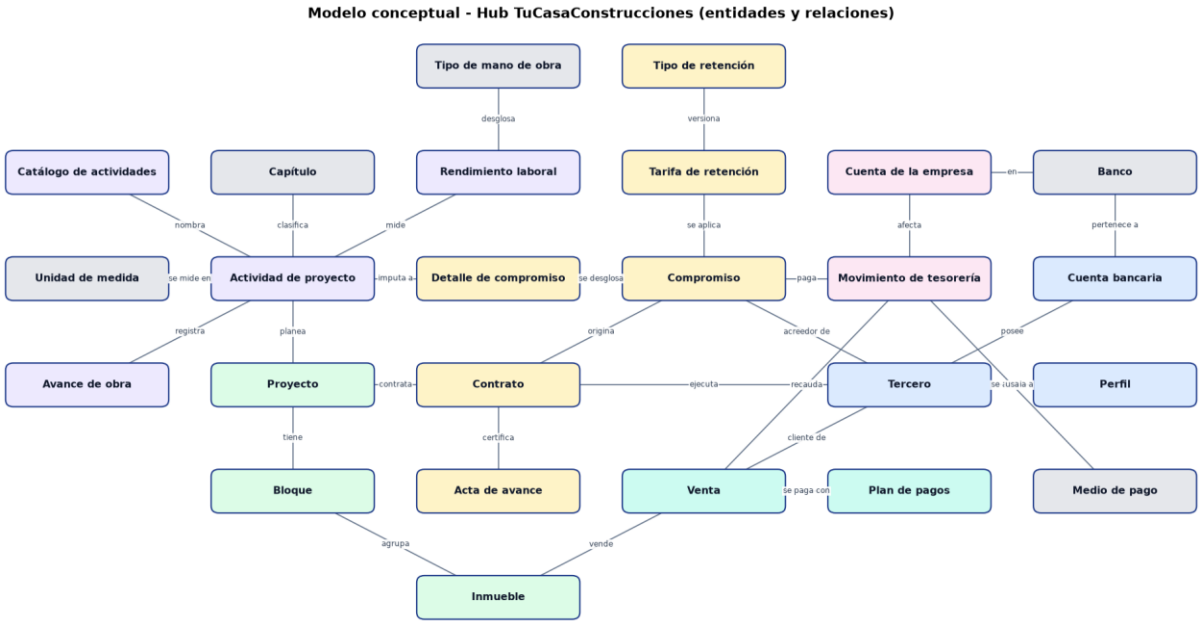


Figura 1. Modelo conceptual

2.6 Modelo lógico (Diagrama de ER)

- El modelo lógico detalla, para cada entidad, sus atributos, identificadores (#), atributos obligatorios (*), opcionales (o) y únicos (U), así como la cardinalidad de las relaciones. Se construyó a partir del esquema real desplegado en Supabase (proyecto base_de_datos, PostgreSQL 17) y contiene 29 tablas.
- La normalización se aplicó hasta la tercera forma normal con el siguiente resultado:

1FN: Todos los atributos son atómicos y cada tabla tiene una llave primaria (simple de tipo uuid en 27 tablas y compuesta en 2). Los grupos repetitivos se separaron en tablas hijas: los ítems de una factura en commitment_details, las cuotas de una venta en payment_plans y los contratistas y el equipo de un rendimiento en labor_performance_contractors y labor_performance_crew.

2FN: En las tablas con llave compuesta (labor_performance_crew y labor_performance_contractors) el único atributo no llave, worker_count, depende de toda la llave. En el resto la llave es un identificador simple, por lo que no hay dependencias parciales.

3FN: Los valores repetidos en varios módulos se extrajeron a catálogos: banks, payment_methods, chapters, units, labor_types, retention_types y activity_catalog. Las tarifas de retención se versionan en retention_type_rates para que un cambio de ley no altere facturas anteriores. Los valores derivados (invoice_total, net_payable, budgeted_cost, actual_cost, total_price de los ítems y credit_amount) son columnas generadas por la base de datos, y los saldos y estados (pagado, avance, cuota) no se almacenan: se calculan con consultas, de modo que cada dato se guarda en un solo lugar.

Desnormalización deliberada: properties y project_activities guardan project_id además de block_id para garantizar, con una llave foránea compuesta (project_id, block_id) hacia blocks, que el bloque pertenece al mismo proyecto. Es una redundancia controlada.

- Análisis CRUD (C: crear, R: leer, U: actualizar, D: eliminar). Cada función del negocio tiene su entidad en el modelo y su pantalla en el sistema:

Módulo (pantalla)	Entidades principales	C	R	U	D	Observación
Proyectos (/projects)	projects, blocks, properties	X	X	X	-	Un inmueble solo se asigna a un bloque housing del mismo proyecto.
Terceros (/suppliers, /workers, /partners, /users)	users, bank_accounts	X	X	X	-	Tipo y número de documento obligatorios; un tercero con movimientos no se elimina (llaves foráneas).
Compromisos (/commitments)	commitments, commitment_details, retention_type_rates	X	X	X	X	Total y neto a pagar los calcula la base de datos.
Contratos (/contracts)	contracts, contract_advances	X	X	X	-	Solo contratistas; estados: active, suspended, pending_closure y closed.
Ventas (en proyectos)	sales, payment_plans, payment_applications	X	X	X	-	El estado de la venta sincroniza el del inmueble; los pagos se aplican a las cuotas más antiguas primero.
Tesorería (/financial)	treasury_movements, company_accounts, commitment_applications	X	X	-	-	Libro inmutable: monto mayor que 0; los errores se corrigen con un movimiento contrario.
Obras y Control (/budgets, /timelines, /work-progress, /labor-performance)	project_activities, activity_catalog, work_progress, labor_performance, labor_types	X	X	X	X	Presupuesto, cronograma, avance y rendimiento de personal.
Catálogos (/admin)	banks, payment_methods, chapters, units, retention_types	X	X	X	-	Nombres y códigos únicos; se desactivan con is_active cuando la tabla lo tiene.
Acceso (/admin, /platform)	profiles	X	X	X	-	Roles: superadmin, admin, accountant, foreman y assistant.

- El modelo lógico se contrastó con el esquema real de la copia en Supabase mediante consultas al catálogo del sistema (pg_class, pg_attribute, pg_constraint, pg_proc y pg_trigger), de modo que sus entidades, atributos, llaves y restricciones coinciden con la base desplegada.

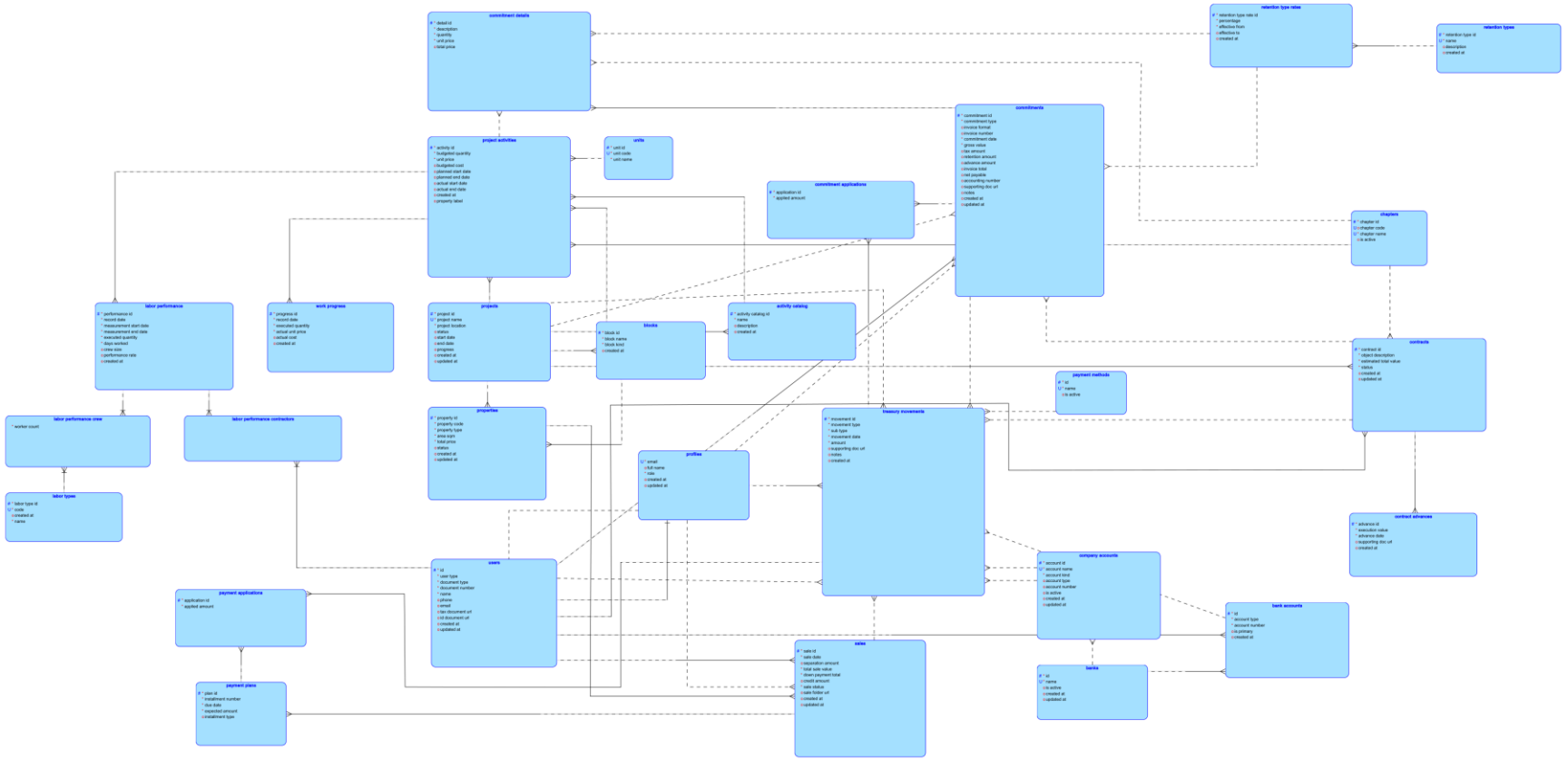


Figura 2. Modelo lógico

3. Diseño

3.1 Especificaciones de diseño

El diseño físico se deriva del modelo lógico y se implementa tal como está desplegado en Supabase (PostgreSQL 17). Las decisiones de diseño son:

- Llaves primarias uuid generadas con gen_random_uuid(), excepto profiles (cuyo id es el del usuario de autenticación) y las dos tablas intermedias de rendimiento, que usan llave compuesta.
- Dominios controlados (estados, tipos y roles) con restricciones CHECK, en lugar de tablas de tipos, porque son pocos valores y estables; los montos se controlan con CHECK (mayor que 0 o mayor o igual a 0).
- Valores derivados como columnas generadas (GENERATED ALWAYS AS … STORED): invoice_total, net_payable, total_price de los ítems, budgeted_cost, actual_cost y credit_amount.
- Llaves foráneas compuestas (project_id, block_id) hacia blocks en properties y project_activities, para garantizar que el bloque pertenece al mismo proyecto.
- Trazabilidad con created_at, updated_at y created_by (perfil que crea) en las tablas transaccionales principales.
- Reglas de procedimiento resueltas en la base con cinco funciones y cinco triggers; los saldos y estados no se almacenan, se calculan con consultas.

Tabla 1. Resumen de las 29 tablas por módulo, con el número de filas poblado en la base (datos sintéticos).

Módulo	Tabla	Descripción	Filas
Terceros y acceso	profiles	Perfiles de acceso, uno por usuario autenticado, con su rol	15
Terceros y acceso	users	Terceros: clientes, proveedores, contratistas y empleados	75
Terceros y acceso	bank_accounts	Cuentas bancarias de los terceros	40
Catálogos	banks	Bancos	15
Catálogos	payment_methods	Medios de pago	15
Catálogos	chapters	Capítulos del presupuesto	15
Catálogos	units	Unidades de medida	15
Catálogos	retention_types	Tipos de retención	15
Catálogos	retention_type_rates	Tarifas porcentuales de retención, versionadas por vigencia	15
Catálogos	labor_types	Tipos de mano de obra	15
Proyectos	projects	Proyectos inmobiliarios	500
Proyectos	blocks	Bloques de un proyecto (viviendas o zonas comunes)	1000
Proyectos	properties	Inmuebles (unidades de vivienda) de un proyecto	500
Obras y control	activity_catalog	Catálogo de actividades por proyecto	500
Obras y control	project_activities	Actividades presupuestadas por proyecto, bloque y capítulo	1000
Obras y control	work_progress	Avance de obra de cada actividad	40
Obras y control	labor_performance	Rendimiento del personal por actividad	20
Obras y control	labor_performance_contractors	Contratistas que participan en un rendimiento (tabla intermedia)	20
Obras y control	labor_performance_crew	Cuadrilla por tipo de mano de obra de un rendimiento (tabla intermedia)	20
Compromisos y contratos	contracts	Contratos con contratistas	20
Compromisos y contratos	contract_advances	Actas de avance de los contratos	15

Módulo	Tabla	Descripción	Filas
Compromisos y contratos	commitments	Compromisos de pago: facturas, cuentas de cobro y liquidaciones	500
Compromisos y contratos	commitment_details	Ítems de un compromiso	968
Ventas	sales	Ventas de inmuebles a clientes	200
Ventas	payment_plans	Cuotas del plan de pagos de cada venta	600
Tesorería	company_accounts	Cuentas de la empresa (banco, caja menor, efectivo e inversión)	15
Tesorería	treasury_movements	Movimientos de tesorería (ingresos, egresos, traslados e inversiones)	500
Tesorería	commitment_applications	Aplicación de movimientos a compromisos (tabla intermedia)	150
Tesorería	payment_applications	Aplicación de movimientos a cuotas de venta (tabla intermedia)	250

Diccionario de datos de las 29 tablas (opcionalidad: * obligatorio, o opcional; pk: llave primaria, fk: llave foránea, uk: valor único):

Tabla 2. profiles (perfiles de acceso, uno por usuario autenticado, con su rol)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk, fk	*	id	UUID	-> auth.users.id
uk	*	email	VARCHAR	
	o	full_name	VARCHAR	
	*	role	VARCHAR	superadmin, admin, accountant, foreman, assistant
	*	created_at	TIMESTAMPTZ	
	*	updated_at	TIMESTAMPTZ	

Tabla 3. users (terceros: clientes, proveedores, contratistas y empleados)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	id	UUID	
	*	user_type	VARCHAR	client, provider, contractor, employee
	*	document_type	VARCHAR	CC, NIT, CE, PASSPORT
	*	document_number	VARCHAR	
	*	name	VARCHAR	
	o	phone	VARCHAR	
	o	email	VARCHAR	
	o	tax_document_url	TEXT	
	o	id_document_url	TEXT	
fk, uk	o	profile_id	UUID	-> profiles.id
	*	created_at	TIMESTAMPTZ	
	*	updated_at	TIMESTAMPTZ	

Tabla 4. bank_accounts (cuentas bancarias de los terceros)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	id	UUID	
fk	*	user_id	UUID	-> users.id
fk	*	bank_id	UUID	-> banks.id
	*	account_type	VARCHAR	savings, checking, digital_wallet
	*	account_number	VARCHAR	
	*	is_primary	BOOLEAN	por defecto false
	*	created_at	TIMESTAMPTZ	

Tabla 5. banks (bancos)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	id	UUID	
uk	*	name	VARCHAR	

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
	*	is_active	BOOLEAN	por defecto true
	*	created_at	TIMESTAMPTZ	
	*	updated_at	TIMESTAMPTZ	

Tabla 6. payment_methods (medios de pago)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	id	UUID	
uk	*	name	VARCHAR	
	*	is_active	BOOLEAN	por defecto true

Tabla 7. chapters (capítulos del presupuesto)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	chapter_id	UUID	
uk	o	chapter_code	VARCHAR	
uk	*	chapter_name	VARCHAR	
	*	is_active	BOOLEAN	por defecto true

Tabla 8. units (unidades de medida)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	unit_id	UUID	
uk	*	unit_code	VARCHAR	
	*	unit_name	VARCHAR	

Tabla 9. retention_types (tipos de retención)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	retention_type_id	UUID	
uk	*	name	VARCHAR	
	o	description	TEXT	
	*	created_at	TIMESTAMPTZ	

Tabla 10. retention_type_rates (tarifas porcentuales de retención, versionadas por vigencia)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	retention_type_rate_id	UUID	
fk	*	retention_type_id	UUID	-> retention_types.retention_type_id
	*	percentage	NUMERIC	0 a 100
	*	effective_from	DATE	
	o	effective_to	DATE	
	*	created_at	TIMESTAMPTZ	

Tabla 11. labor_types (tipos de mano de obra)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	labor_type_id	UUID	
uk	*	code	VARCHAR	formato ^[a-z][a-z0-9_]*$
	*	created_at	TIMESTAMPTZ	
	*	name	VARCHAR	

Tabla 12. projects (proyectos inmobiliarios)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	project_id	UUID	
uk	*	project_name	VARCHAR	
	*	project_location	TEXT	
	*	status	VARCHAR	active, completed, on_hold | por defecto 'active'

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
	o	start_date	DATE	
	o	end_date	DATE	
	*	progress	INTEGER	0 a 100 | por defecto 0
	*	created_at	TIMESTAMPTZ	
	*	updated_at	TIMESTAMPTZ	

Tabla 13. blocks (bloques de un proyecto (viviendas o zonas comunes))

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	block_id	UUID	
fk	*	project_id	UUID	-> projects.project_id
	*	block_name	VARCHAR	
	*	block_kind	VARCHAR	housing, common_areas
	*	created_at	TIMESTAMPTZ	

Restricción única compuesta: (project_id, block_id).

Tabla 14. properties (inmuebles (unidades de vivienda) de un proyecto)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	property_id	UUID	
fk	*	project_id	UUID	-> projects.project_id; blocks.project_id
fk	*	block_id	UUID	-> blocks.block_id
	*	property_code	VARCHAR	
	*	property_type	VARCHAR	
	*	area_sqm	NUMERIC	> 0
	*	total_price	NUMERIC	>= 0
	*	status	VARCHAR	available, reserved, sold | por defecto 'available'
	*	created_at	TIMESTAMPTZ	
	*	updated_at	TIMESTAMPTZ	

Tabla 15. activity_catalog (catálogo de actividades por proyecto)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	activity_catalog_id	UUID	
	*	name	VARCHAR	
	o	description	TEXT	
	*	created_at	TIMESTAMPTZ	
fk	*	project_id	UUID	-> projects.project_id

Tabla 16. project_activities (actividades presupuestadas por proyecto, bloque y capítulo)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	activity_id	UUID	
fk	*	project_id	UUID	-> projects.project_id; blocks.project_id
fk	*	chapter_id	UUID	-> chapters.chapter_id
fk	*	block_id	UUID	-> blocks.block_id
fk	*	activity_catalog_id	UUID	-> activity_catalog.activity_catalog_id
fk	*	unit_id	UUID	-> units.unit_id
	*	budgeted_quantity	NUMERIC	> 0
	*	unit_price	NUMERIC	>= 0
	o	budgeted_cost	NUMERIC	calculado: budgeted_quantity x unit_price
	o	planned_start_date	DATE	
	o	planned_end_date	DATE	
	o	actual_start_date	DATE	
	o	actual_end_date	DATE	
	*	created_at	TIMESTAMPTZ	
	o	property_label	VARCHAR	

Tabla 17. work_progress (avance de obra de cada actividad)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	progress_id	UUID	
fk	*	activity_id	UUID	-> project_activities.activity_id
	*	record_date	DATE	
	*	executed_quantity	NUMERIC	> 0
	*	actual_unit_price	NUMERIC	>= 0
	o	actual_cost	NUMERIC	calculado: executed_quantity x actual_unit_price
	*	created_at	TIMESTAMPTZ	

Tabla 18. labor_performance (rendimiento del personal por actividad)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	performance_id	UUID	
fk	*	activity_id	UUID	-> project_activities.activity_id
	*	record_date	DATE	
	*	measurement_start_date	DATE	
	*	measurement_end_date	DATE	
	*	executed_quantity	NUMERIC	> 0
	*	days_worked	INTEGER	> 0
	*	crew_size	INTEGER	>= 0 | por defecto 0
	o	performance_rate	NUMERIC	
	*	created_at	TIMESTAMPTZ	

Tabla 19. labor_performance_contractors (contratistas que participan en un rendimiento (tabla intermedia))

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk, fk	*	performance_id	UUID	-> labor_performance.performance_id
pk, fk	*	user_id	UUID	-> users.id

Llave primaria compuesta: (performance_id, user_id).

Tabla 20. labor_performance_crew (cuadrilla por tipo de mano de obra de un rendimiento (tabla intermedia))

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk, fk	*	performance_id	UUID	-> labor_performance.performance_id
pk, fk	*	labor_type_id	UUID	-> labor_types.labor_type_id
	*	worker_count	INTEGER	> 0

Llave primaria compuesta: (performance_id, labor_type_id).

Tabla 21. contracts (contratos con contratistas)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	contract_id	UUID	
fk	*	contractor_id	UUID	-> users.id
fk	*	project_id	UUID	-> projects.project_id
fk	o	chapter_id	UUID	-> chapters.chapter_id
	*	object_description	TEXT	
	*	estimated_total_value	NUMERIC	> 0
	*	status	VARCHAR	active, suspended, pending_closure, closed
	*	created_at	TIMESTAMPTZ	
	*	updated_at	TIMESTAMPTZ	

Tabla 22. contract_advances (actas de avance de los contratos)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	advance_id	UUID	
fk	*	contract_id	UUID	-> contracts.contract_id
	*	execution_value	NUMERIC	>= 0

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
	*	advance_date	DATE	
	o	supporting_doc_url	TEXT	
	*	created_at	TIMESTAMPTZ	

Tabla 23. commitments (compromisos de pago: facturas, cuentas de cobro y liquidaciones)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	commitment_id	UUID	
fk	o	project_id	UUID	-> projects.project_id
fk	*	user_id	UUID	-> users.id
fk	o	contract_id	UUID	-> contracts.contract_id
	*	commitment_type	VARCHAR	invoice, collection_account, contract_settlement
	o	invoice_format	VARCHAR	physical, electronic
	o	invoice_number	VARCHAR	
	*	commitment_date	DATE	
	*	gross_value	NUMERIC	>= 0
	*	tax_amount	NUMERIC	>= 0 | por defecto 0.00
	*	retention_amount	NUMERIC	>= 0 | por defecto 0.00
fk	o	retention_type_rate_id	UUID	-> retention_type_rates.retention_type_rate_id
	*	advance_amount	NUMERIC	>= 0 | por defecto 0
	o	invoice_total	NUMERIC	calculado: gross_value + tax_amount
	o	net_payable	NUMERIC	calculado: ((gross_value + tax_amount) - retention_amount) - advance_amount
	o	accounting_number	VARCHAR	
	o	supporting_doc_url	TEXT	
	o	notes	TEXT	
	*	created_at	TIMESTAMPTZ	
	*	updated_at	TIMESTAMPTZ	
fk	*	created_by	UUID	por defecto auth.uid() | -> profiles.id

Tabla 24. commitment_details (ítems de un compromiso)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	detail_id	UUID	
fk	*	commitment_id	UUID	-> commitments.commitment_id
fk	o	activity_id	UUID	-> project_activities.activity_id
fk	o	chapter_id	UUID	-> chapters.chapter_id
	*	description	TEXT	
	*	quantity	NUMERIC	> 0
	*	unit_price	NUMERIC	>= 0
	o	total_price	NUMERIC	calculado: quantity x unit_price
fk	o	retention_type_rate_id	UUID	-> retention_type_rates.retention_type_rate_id

Tabla 25. sales (ventas de inmuebles a clientes)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	sale_id	UUID	
fk	*	customer_id	UUID	-> users.id
fk	*	property_id	UUID	-> properties.property_id
	*	sale_date	DATE	
	*	separation_amount	NUMERIC	>= 0 | por defecto 0.00
	*	total_sale_value	NUMERIC	> 0
	*	down_payment_total	NUMERIC	
	o	credit_amount	NUMERIC	calculado: total_sale_value - down_payment_total
	*	sale_status	VARCHAR	active, completed, canceled
	o	sale_folder_url	TEXT	
	*	created_at	TIMESTAMPTZ	
	*	updated_at	TIMESTAMPTZ	
fk	*	created_by	UUID	por defecto auth.uid() | -> profiles.id

Tabla 26. payment_plans (cuotas del plan de pagos de cada venta)

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	plan_id	UUID	
fk	*	sale_id	UUID	-> sales.sale_id
	*	installment_number	INTEGER	> 0
	*	due_date	DATE	
	*	expected_amount	NUMERIC	> 0
	*	installment_type	VARCHAR	down_payment_installment, mortgage_credit, subsidy, extra | por defecto 'down_payment_installment'

Tabla 27. company_accounts (cuentas de la empresa (banco, caja menor, efectivo e inversión))

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	account_id	UUID	
uk	*	account_name	VARCHAR	
	*	account_kind	VARCHAR	bank, petty_cash, cash, investment
fk	o	bank_id	UUID	-> banks.id
	o	account_type	VARCHAR	savings, checking, digital_wallet
	o	account_number	VARCHAR	
	*	is_active	BOOLEAN	por defecto true
	*	created_at	TIMESTAMPTZ	
	*	updated_at	TIMESTAMPTZ	

Tabla 28. treasury_movements (movimientos de tesorería (ingresos, egresos, traslados e inversiones))

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	movement_id	UUID	
	*	movement_type	VARCHAR	income, expense, transfer, investment
	*	sub_type	VARCHAR	sale_payment, sale_refund, contract_advance, commitment_payment, petty_cash_funding, petty_cash_expense, internal_transfer, cdt, loan
	*	movement_date	DATE	
	*	amount	NUMERIC	> 0
fk	o	project_id	UUID	-> projects.project_id
fk	o	user_id	UUID	-> users.id
fk	o	commitment_id	UUID	-> commitments.commitment_id
fk	o	sale_id	UUID	-> sales.sale_id
fk	o	contract_id	UUID	-> contracts.contract_id
fk	o	payment_method_id	UUID	-> payment_methods.id
fk	o	source_account_id	UUID	-> company_accounts.account_id
fk	o	dest_account_id	UUID	-> company_accounts.account_id
fk	o	dest_bank_account_id	UUID	-> bank_accounts.id
	o	supporting_doc_url	TEXT	
	o	notes	TEXT	
fk	*	created_by	UUID	-> profiles.id
	*	created_at	TIMESTAMPTZ	

Tabla 29. commitment_applications (aplicación de movimientos a compromisos (tabla intermedia))

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	application_id	UUID	
fk	*	movement_id	UUID	-> treasury_movements.movement_id
fk	*	commitment_id	UUID	-> commitments.commitment_id
	*	applied_amount	NUMERIC	> 0

Tabla 30. payment_applications (aplicación de movimientos a cuotas de venta (tabla intermedia))

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
pk	*	application_id	UUID	
fk	*	movement_id	UUID	-> treasury_movements.movement_id
fk	*	plan_id	UUID	-> payment_plans.plan_id

Tipo de clave	Opcionalidad	Nombre de la columna	Tipo de datos	Valores / tamaño
	*	applied_amount	NUMERIC	> 0

3.2 Transformación del modelo lógico a Diagrama de tablas.

El diseño se derivó del modelo lógico: cada entidad se transformó en una tabla del esquema public; cada relación uno a muchos en una llave foránea; cada relación muchos a muchos en una tabla intermedia (commitment_applications, payment_applications, labor_performance_contractors y labor_performance_crew); cada identificador en una llave primaria y cada atributo único en una restricción UNIQUE. El diagrama de tablas resultante es la Figura 3 (sección 4.1). Las reglas del negocio de la sección 2.3 y las reglas del levantamiento inicial con la empresa se contrastaron con el esquema implementado:

Regla	Cómo se resuelve en el modelo	Estado
1	users.user_type y users.document_type con CHECK; users.profile_id UNIQUE. No existe UNIQUE (document_type, document_number).	Parcial
2	projects.project_name UNIQUE; CHECK en status y en progress (0 a 100).	Cumple
3	blocks.block_kind con CHECK (housing, common_areas); UNIQUE (project_id, block_id).	Cumple
4	Llave foránea compuesta properties (project_id, block_id) hacia blocks; CHECK en area_sqm > 0, total_price >= 0 y status.	Cumple
5	Trigger trg_property_block_must_be_housing (función fn_property_block_must_be_housing).	Cumple
6	Trigger trg_sync_property_status_with_sale (función fn_sync_property_status_with_sale).	Cumple
7	sales.customer_id sin restricción única (un cliente puede comprar varios inmuebles); credit_amount es columna generada.	Cumple
7b	Una sola venta vigente por inmueble: no existe un índice único parcial sobre sales (property_id).	Por completar
8	payment_plans con CHECK en installment_number > 0, expected_amount > 0 e installment_type.	Cumple
9	Trigger trg_apply_sale_payment_fifo (función fn_apply_sale_payment_fifo) inserta en payment_applications.	Cumple
10	commitments con CHECK en commitment_type, invoice_format y en los valores (>= 0).	Cumple
11	commitments.invoice_total y net_payable son columnas generadas.	Cumple
12	commitment_details con CHECK (quantity > 0, unit_price >= 0); total_price generado.	Cumple
13	retention_type_rates con CHECK de porcentaje (0 a 100) y fechas de vigencia; la base no impide vigencias traslapadas.	Parcial
14	contracts con CHECK en estimated_total_value > 0 y status; contract_advances.execution_value >= 0.	Cumple
15	Trigger trg_contract_contractor_must_be_contractor (función fn_contract_contractor_must_be_contractor).	Cumple
16	treasury_movements con CHECK en amount > 0, movement_type y sub_type; created_by NOT NULL con llave foránea a profiles.	Cumple
17	company_accounts.account_name UNIQUE; CHECK en account_kind y account_type.	Cumple
18	project_activities con llave foránea compuesta (project_id, block_id); CHECK de cantidad y precio; budgeted_cost generado.	Cumple
19	Trigger trg_projects_set_updated_at solo en la tabla projects; en las demás tablas updated_at no se actualiza automáticamente.	Parcial
20	UNIQUE en banks.name, payment_methods.name, chapters.chapter_name y chapter_code, units.unit_code, labor_types.code y retention_types.name.	Cumple
C1	Política de entrega "obra gris habitable" (levantamiento inicial): no existe una columna ni una tabla que la registre.	Por completar
C2	Observaciones comerciales por cliente: no existe un campo ni una tabla.	Por completar
C3	Mora a más de 30 días y bloqueo del acta de entrega: no existe tabla de acta de entrega; el atraso se puede calcular con payment_plans.due_date y payment_applications.	Por completar
C4	No registrar escrituración ni entrega de llaves sin el pago total: no está modelada.	Por completar
C5	Recibo PDF automático por abono: treasury_movements.supporting_doc_url guarda el soporte, pero la generación automática no está en la base.	Parcial
C6	Tipología con precio base: properties.property_type (texto), area_sqm y total_price describen la tipología; no existe una tabla de tipologías.	Parcial

Gráfico de instancias de tabla: muestra de 5 filas reales de las tablas principales de la base desplegada (datos sintéticos generados con IA), consultada el 08/11/2026. El identificador (uuid) se abrevia a sus primeros 8 caracteres.

Tabla 31. Instancias de users (terceros)

id	user_type	document_type	document_number	name	phone	email
054ef1b5…	client	CC	1000174218	María Ramírez	3100000814	user22@tucasa.test
0b8f8acf…	contractor	NIT	1000395950	Daniela Rojas	3100001850	user50@tucasa.test

id	user_type	document_type	document_number	name	phone	email
0cee34eb…	provider	NIT	1000578087	Luis Martínez	3100002701	user73@tucasa.test
1000d134…	employee	CC	1000071271	Sergio Ramírez	3100000333	user9@tucasa.test
219510c1…	provider	NIT	1000506816	Laura Ortiz	3100002368	user64@tucasa.test

Tabla 32. Instancias de projects (proyectos)

project_id	project_name	project_location	status	start_date	end_date	progress
0016a062…	Proyecto 0230 Conjunto Residencial Los Robles	Cúcuta, Colombia	active	2023-08-19	2025-01-30	85
00bc93fb…	Proyecto 0148 Edificio Bellavista	Pamplona, Colombia	active	2023-05-29	2024-08-19	51
02219a72…	Proyecto 0462 Urbanización Altos del Valle	Bucaramanga, Colombia	active	2024-04-07	2025-04-04	89
02cf0ab9…	Proyecto 0344 Condominio La Pradera	Villa del Rosario, Colombia	active	2023-12-11	2025-09-15	73
0366d2e6…	Proyecto 0331 Torres de El Bosque	Ocaña, Colombia	active	2023-11-28	2025-08-20	72

Tabla 33. Instancias de properties (inmuebles)

property_id	property_code	property_type	area_sqm	total_price	status
006e6a37…	P-0124-101	Casa	113	452000000	sold
021878e1…	P-0122-101	Local comercial	87	304500000	sold
024c2b67…	P-0194-101	Local comercial	51	204000000	sold
02514a7e…	P-0149-101	Local comercial	114	598500000	sold
02fab76d…	P-0154-101	Casa	98	392000000	sold

Tabla 34. Instancias de sales (ventas)

sale_id	sale_date	separation_amount	total_sale_value	down_payment_total	credit_amount	sale_status
02b84c0e…	2024-04-25	16362500	327250000	98175000	229075000	active
035e2dd9…	2024-03-26	19550000	391000000	117300000	273700000	active
03e3ba6f…	2024-04-14	19200000	384000000	115200000	268800000	active
072f927a…	2024-02-19	28875000	577500000	173250000	404250000	active
082480d7…	2024-02-13	21187500	423750000	127125000	296625000	completed

Tabla 35. Instancias de commitments (compromisos)

commitment_id	invoice_number	commitment_type	commitment_date	gross_value	tax_amount	retention_amount	invoice_total	net_payable
00597a7f…	NULL	collection_account	2024-07-10	4500000	0	157500	4500000	4342500
01146a1b…	FAC-00213	invoice	2024-09-01	5550000	1054500	194250	6604500	6410250
011a3c6c…	FAC-00257	invoice	2024-10-15	3450000	655500	120750	4105500	3984750
013abc96…	FAC-00464	invoice	2024-07-14	5900000	1121000	206500	7021000	6814500
013ed407…	NULL	collection_account	2024-04-20	11150000	0	390250	11150000	10759750

Tabla 36. Instancias de treasury_movements (movimientos de tesorería)

movement_id	movement_date	movement_type	sub_type	amount
00e206c8…	2024-03-31	income	sale_payment	12600000
0149745e…	2024-05-07	expense	petty_cash_expense	67514
0150b2e5…	2024-07-08	income	sale_payment	21000000
01e25438…	2024-06-16	expense	petty_cash_expense	104274
02218530…	2024-07-08	expense	commitment_payment	10279500

La Tabla 37 es la matriz de trazabilidad de la entrega: relaciona cada requisito de la sección 2.2 con las reglas del negocio de la sección 2.3 que lo concretan, las tablas del modelo físico que lo implementan (sección 3.1), los scripts que lo crean y lo pueblan (sección 4.2: esquema_copia.sql y datos_completos.sql, publicados en el repositorio de la sección 4.5), los casos de prueba del plan UAT que lo verifican (sección 5) y la evidencia en la solución desplegada en Supabase (secciones 3.2, 6.1 y 6.2 y Anexo 1).

Tabla 37. Matriz de trazabilidad: requisito, regla del negocio, tabla, script, prueba y evidencia en la base desplegada

Req.	Reglas	Tablas del modelo físico	Script (4.2)	Pruebas UAT	Evidencia en la solución desplegada
1	1	users, bank_accounts, banks	DDL y DML	INT-08	Tabla 31 (instancias de users); conteos de las 29 tablas en el Anexo 1, E7
2	1	profiles, users	DDL y DML	INT-04, SEC-01, SEC-02	Anexo 1, E7 (15 perfiles); hallazgos de seguridad SEC-01 y SEC-02 (sección 5)
3	2, 3, 4, 5	projects, blocks, properties	DDL y DML	INT-03, INT-05, TRG-01	Figuras 5 y 7; Tablas 32 y 33
4	6, 7	sales, properties	DDL y DML	INT-06, TRG-03, QRY-01, QRY-02	Tabla 34; Figura 10 (ventas por proyecto)
5	8, 9	payment_plans, payment_applications	DDL y DML	TRG-04	Anexo 1, E7 (600 cuotas y 250 aplicaciones de pago)
6	10, 11, 12	commitments, commitment_details, commitment_applications	DDL y DML; completar_datos. sql	INT-07, QRY-03, QRY-04	Tabla 35; Figuras 8 y 9; Anexo 1, E4 y E8
7	13	retention_types, retention_type_rates	DDL y DML	Sin prueba dedicada (la regla 13 figura como Parcial en la Tabla de la sección 3.2)	Anexo 1, E7 (15 tipos y 15 tarifas)
8	14, 15	contracts, contract_advances	DDL y DML	TRG-02	Anexo 1, E7 (20 contratos y 15 actas de avance)
9	16, 17	company_accounts, treasury_movements, commitment_applications, payment_applications	DDL y DML	INT-04, TRG-04, QRY-04	Tabla 36; Figura 7 (500 movimientos)
10	18	project_activities, activity_catalog, work_progress, labor_performance, labor_performance_contractors, labor_performance_crew, labor_types	DDL y DML	QRY-05, PER-03	Figura 7 (1000 actividades); Anexo 1, E7
11	20	banks, payment_methods, chapters, units, labor_types, retention_types	DDL y DML	INT-02	Figuras 11, 12 y 13 (INSERT, UPDATE y DELETE sobre banks)
12	19	created_at, updated_at y created_by en las tablas principales	DDL (trigger trg_projects_set_ updated_at)	TRG-05, INT-04	Sección 5 (resultado de TRG-05)
RNF	Concurrencia, rendimiento, recuperación y seguridad (requisitos no funcionales)	Base de datos completa	DDL y DML	CON-01, CON-02, PER-01 a PER-03, REC-01, REC-02, SEC-01 a SEC-03	Figura 4 (proyecto Healthy en Supabase) y secciones 5 y 6

4. Creación

4.1 Modelo físico.

El modelo físico se implementó en PostgreSQL 17 sobre Supabase (proyecto base_de_datos, región us-east-1). Cada entidad del modelo lógico es una tabla del esquema public, con llave primaria uuid (gen_random_uuid()), restricciones CHECK para los dominios controlados (estados, tipos y montos), restricciones UNIQUE para los atributos únicos, llaves foráneas (incluidas dos llaves compuestas (project_id, block_id) hacia blocks) y valores por defecto. Los valores derivados son columnas generadas (invoice_total, net_payable, total_price de los ítems, budgeted_cost, actual_cost y credit_amount). La lógica de procedimiento se resuelve con cinco funciones y cinco triggers. Esta versión no define vistas, políticas RLS ni extensiones adicionales a las de Supabase (plpgsql, pg_stat_statements, uuid-ossp, pgcrypto y supabase_vault): los saldos y estados se obtienen con consultas y el control de permisos por rol se aplica en la aplicación (ver recomendaciones).

Tabla de triggers y funciones implementados en la base de datos:

Trigger	Tabla y evento	Función	Regla que implementa
trg_projects_set_updated_at	projects, BEFORE UPDATE	fn_set_updated_at	Actualiza updated_at (regla 19)
trg_property_block_must_be_housing	properties, BEFORE INSERT o UPDATE de block_id y project_id	fn_property_block_must_be_housing	El inmueble solo va en un bloque housing (regla 5)
trg_contract_contractor_must_be_contractor	contracts, BEFORE INSERT o UPDATE de contractor_id	fn_contract_contractor_must_be_contractor	El contratista debe ser de tipo contractor (regla 15)
trg_sync_property_status_with_sale	sales, AFTER INSERT o UPDATE de sale_status	fn_sync_property_status_with_sale	Sincroniza el estado del inmueble con la venta (regla 6)
trg_apply_sale_payment_fifo	treasury_movements, AFTER INSERT	fn_apply_sale_payment_fifo	Aplica el pago a las cuotas más antiguas primero (regla 9)

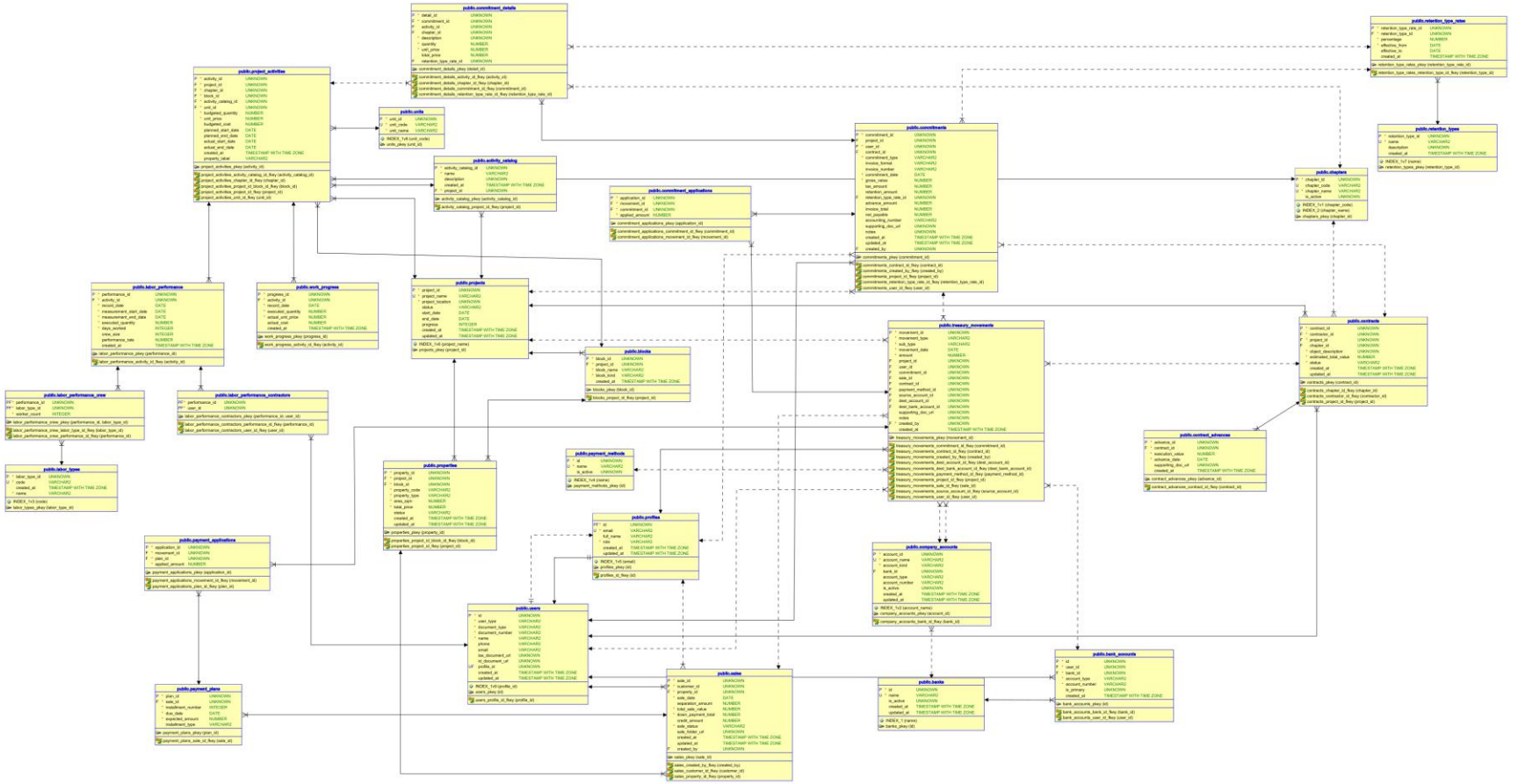


Figura 3. Modelo físico

4.2 Script DDL y DML

Script DDL (estructura)

El script completo se incluye a continuación y se entrega también como archivo adjunto (esquema_copia.sql). Se creó respetando el orden de dependencias de las llaves foráneas: catálogos, acceso y terceros, proyectos, obras y control, contratos y compromisos, ventas y tesorería; al final se crean las funciones y los triggers. profiles referencia auth.users, que existe en Supabase (esquema auth).

```
-- ---------- Catalogos ----------
CREATE TABLE public.banks (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  name character varying NOT NULL,
  is_active boolean NOT NULL DEFAULT true,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  updated_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT banks_pkey PRIMARY KEY (id),
  CONSTRAINT banks_name_key UNIQUE (name)
);

CREATE TABLE public.payment_methods (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  name character varying NOT NULL,
  is_active boolean NOT NULL DEFAULT true,
  CONSTRAINT payment_methods_pkey PRIMARY KEY (id),
  CONSTRAINT payment_methods_name_key UNIQUE (name)
);

CREATE TABLE public.chapters (
  chapter_id uuid NOT NULL DEFAULT gen_random_uuid(),
  chapter_code character varying,
  chapter_name character varying NOT NULL,
  is_active boolean NOT NULL DEFAULT true,
  CONSTRAINT chapters_pkey PRIMARY KEY (chapter_id),
  CONSTRAINT chapters_chapter_code_key UNIQUE (chapter_code),
  CONSTRAINT chapters_chapter_name_key UNIQUE (chapter_name)
);

CREATE TABLE public.units (
  unit_id uuid NOT NULL DEFAULT gen_random_uuid(),
  unit_code character varying NOT NULL,
  unit_name character varying NOT NULL,
  CONSTRAINT units_pkey PRIMARY KEY (unit_id),
  CONSTRAINT units_unit_code_key UNIQUE (unit_code)
);

CREATE TABLE public.retention_types (
  retention_type_id uuid NOT NULL DEFAULT gen_random_uuid(),
  name character varying NOT NULL,
  description text,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT retention_types_pkey PRIMARY KEY (retention_type_id),
  CONSTRAINT retention_types_name_key UNIQUE (name)
);

CREATE TABLE public.retention_type_rates (
  retention_type_rate_id uuid NOT NULL DEFAULT gen_random_uuid(),
  retention_type_id uuid NOT NULL,
  percentage numeric NOT NULL,
  effective_from date NOT NULL,
  effective_to date,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT retention_type_rates_pkey PRIMARY KEY (retention_type_rate_id),
  CONSTRAINT retention_type_rates_retention_type_id_fkey FOREIGN KEY (retention_type_id) REFERENCES
public.retention_types(retention_type_id),
  CONSTRAINT retention_type_rates_percentage_check CHECK (percentage >= 0 AND percentage <= 100)
);

CREATE TABLE public.labor_types (
  labor_type_id uuid NOT NULL DEFAULT gen_random_uuid(),
  code character varying NOT NULL,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  name character varying NOT NULL,
```

```
  CONSTRAINT labor_types_pkey PRIMARY KEY (labor_type_id),
  CONSTRAINT labor_types_code_key UNIQUE (code),
  CONSTRAINT labor_types_code_check CHECK (code::text ~ '^[a-z][a-z0-9_]*$')
);

-- ---------- Acceso y terceros ----------
CREATE TABLE public.profiles (
  id uuid NOT NULL,
  email character varying NOT NULL,
  full_name character varying,
  role character varying NOT NULL,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  updated_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT profiles_pkey PRIMARY KEY (id),
  CONSTRAINT profiles_email_key UNIQUE (email),
  CONSTRAINT profiles_id_fkey FOREIGN KEY (id) REFERENCES auth.users(id),
  CONSTRAINT profiles_role_check CHECK (role::text = ANY (ARRAY['superadmin','admin','accountant','foreman','assistant']))
);

CREATE TABLE public.users (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  user_type character varying NOT NULL,
  document_type character varying NOT NULL,
  document_number character varying NOT NULL,
  name character varying NOT NULL,
  phone character varying,
  email character varying,
  tax_document_url text,
  id_document_url text,
  profile_id uuid,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  updated_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT users_pkey PRIMARY KEY (id),
  CONSTRAINT users_profile_id_key UNIQUE (profile_id),
  CONSTRAINT users_profile_id_fkey FOREIGN KEY (profile_id) REFERENCES public.profiles(id),
  CONSTRAINT users_user_type_check CHECK (user_type::text = ANY (ARRAY['client','provider','contractor','employee'])),
  CONSTRAINT users_document_type_check CHECK (document_type::text = ANY (ARRAY['CC','NIT','CE','PASSPORT']))
);

CREATE TABLE public.bank_accounts (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL,
  bank_id uuid NOT NULL,
  account_type character varying NOT NULL,
  account_number character varying NOT NULL,
  is_primary boolean NOT NULL DEFAULT false,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT bank_accounts_pkey PRIMARY KEY (id),
  CONSTRAINT bank_accounts_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.users(id),
  CONSTRAINT bank_accounts_bank_id_fkey FOREIGN KEY (bank_id) REFERENCES public.banks(id),
  CONSTRAINT bank_accounts_account_type_check CHECK (account_type::text = ANY (ARRAY['savings','checking','digital_wallet']))
);

-- ---------- Proyectos ----------
CREATE TABLE public.projects (
  project_id uuid NOT NULL DEFAULT gen_random_uuid(),
  project_name character varying NOT NULL,
  project_location text NOT NULL,
  status character varying NOT NULL DEFAULT 'active',
  start_date date,
  end_date date,
  progress integer NOT NULL DEFAULT 0,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  updated_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT projects_pkey PRIMARY KEY (project_id),
  CONSTRAINT projects_project_name_key UNIQUE (project_name),
  CONSTRAINT projects_status_check CHECK (status::text = ANY (ARRAY['active','completed','on_hold'])),
  CONSTRAINT projects_progress_check CHECK (progress >= 0 AND progress <= 100)
);

CREATE TABLE public.blocks (
  block_id uuid NOT NULL DEFAULT gen_random_uuid(),
  project_id uuid NOT NULL,
  block_name character varying NOT NULL,
  block_kind character varying NOT NULL,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT blocks_pkey PRIMARY KEY (block_id),
  CONSTRAINT blocks_project_id_block_id_key UNIQUE (project_id, block_id),
  CONSTRAINT blocks_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(project_id),
  CONSTRAINT blocks_block_kind_check CHECK (block_kind::text = ANY (ARRAY['housing','common_areas']))
);
```

```
CREATE TABLE public.activity_catalog (
  activity_catalog_id uuid NOT NULL DEFAULT gen_random_uuid(),
  name character varying NOT NULL,
  description text,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  project_id uuid NOT NULL,
  CONSTRAINT activity_catalog_pkey PRIMARY KEY (activity_catalog_id),
  CONSTRAINT activity_catalog_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(project_id)
);

CREATE TABLE public.properties (
  property_id uuid NOT NULL DEFAULT gen_random_uuid(),
  project_id uuid NOT NULL,
  block_id uuid NOT NULL,
  property_code character varying NOT NULL,
  property_type character varying NOT NULL,
  area_sqm numeric NOT NULL,
  total_price numeric NOT NULL,
  status character varying NOT NULL DEFAULT 'available',
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  updated_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT properties_pkey PRIMARY KEY (property_id),
  CONSTRAINT properties_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(project_id),
  CONSTRAINT properties_project_id_block_id_fkey FOREIGN KEY (project_id, block_id) REFERENCES public.blocks(project_id,
block_id),
  CONSTRAINT properties_area_sqm_check CHECK (area_sqm > 0),
  CONSTRAINT properties_total_price_check CHECK (total_price >= 0),
  CONSTRAINT properties_status_check CHECK (status::text = ANY (ARRAY['available','reserved','sold']))
);

-- ---------- Obras y control ----------
CREATE TABLE public.project_activities (
  activity_id uuid NOT NULL DEFAULT gen_random_uuid(),
  project_id uuid NOT NULL,
  chapter_id uuid NOT NULL,
  block_id uuid NOT NULL,
  activity_catalog_id uuid NOT NULL,
  unit_id uuid NOT NULL,
  budgeted_quantity numeric NOT NULL,
  unit_price numeric NOT NULL,
  budgeted_cost numeric GENERATED ALWAYS AS (budgeted_quantity * unit_price) STORED,
  planned_start_date date,
  planned_end_date date,
  actual_start_date date,
  actual_end_date date,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  property_label character varying,
  CONSTRAINT project_activities_pkey PRIMARY KEY (activity_id),
  CONSTRAINT project_activities_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(project_id),
  CONSTRAINT project_activities_chapter_id_fkey FOREIGN KEY (chapter_id) REFERENCES public.chapters(chapter_id),
  CONSTRAINT project_activities_activity_catalog_id_fkey FOREIGN KEY (activity_catalog_id) REFERENCES
public.activity_catalog(activity_catalog_id),
  CONSTRAINT project_activities_unit_id_fkey FOREIGN KEY (unit_id) REFERENCES public.units(unit_id),
  CONSTRAINT project_activities_project_id_block_id_fkey FOREIGN KEY (project_id, block_id) REFERENCES public.blocks(project_id,
block_id),
  CONSTRAINT project_activities_budgeted_quantity_check CHECK (budgeted_quantity > 0),
  CONSTRAINT project_activities_unit_price_check CHECK (unit_price >= 0)
);

CREATE TABLE public.work_progress (
  progress_id uuid NOT NULL DEFAULT gen_random_uuid(),
  activity_id uuid NOT NULL,
  record_date date NOT NULL,
  executed_quantity numeric NOT NULL,
  actual_unit_price numeric NOT NULL,
  actual_cost numeric GENERATED ALWAYS AS (executed_quantity * actual_unit_price) STORED,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT work_progress_pkey PRIMARY KEY (progress_id),
  CONSTRAINT work_progress_activity_id_fkey FOREIGN KEY (activity_id) REFERENCES public.project_activities(activity_id),
  CONSTRAINT work_progress_executed_quantity_check CHECK (executed_quantity > 0),
  CONSTRAINT work_progress_actual_unit_price_check CHECK (actual_unit_price >= 0)
);

CREATE TABLE public.labor_performance (
  performance_id uuid NOT NULL DEFAULT gen_random_uuid(),
  activity_id uuid NOT NULL,
  record_date date NOT NULL,
  measurement_start_date date NOT NULL,
  measurement_end_date date NOT NULL,
  executed_quantity numeric NOT NULL,
```

```
  days_worked integer NOT NULL,
  crew_size integer NOT NULL DEFAULT 0,
  performance_rate numeric,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT labor_performance_pkey PRIMARY KEY (performance_id),
  CONSTRAINT labor_performance_activity_id_fkey FOREIGN KEY (activity_id) REFERENCES public.project_activities(activity_id),
  CONSTRAINT labor_performance_executed_quantity_check CHECK (executed_quantity > 0),
  CONSTRAINT labor_performance_days_worked_check CHECK (days_worked > 0),
  CONSTRAINT labor_performance_crew_size_check CHECK (crew_size >= 0)
);

CREATE TABLE public.labor_performance_contractors (
  performance_id uuid NOT NULL,
  user_id uuid NOT NULL,
  CONSTRAINT labor_performance_contractors_pkey PRIMARY KEY (performance_id, user_id),
  CONSTRAINT labor_performance_contractors_performance_id_fkey FOREIGN KEY (performance_id) REFERENCES
public.labor_performance(performance_id),
  CONSTRAINT labor_performance_contractors_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.users(id)
);

CREATE TABLE public.labor_performance_crew (
  performance_id uuid NOT NULL,
  labor_type_id uuid NOT NULL,
  worker_count integer NOT NULL,
  CONSTRAINT labor_performance_crew_pkey PRIMARY KEY (performance_id, labor_type_id),
  CONSTRAINT labor_performance_crew_performance_id_fkey FOREIGN KEY (performance_id) REFERENCES
public.labor_performance(performance_id),
  CONSTRAINT labor_performance_crew_labor_type_id_fkey FOREIGN KEY (labor_type_id) REFERENCES public.labor_types(labor_type_id),
  CONSTRAINT labor_performance_crew_worker_count_check CHECK (worker_count > 0)
);

-- ---------- Contratos y compromisos ----------
CREATE TABLE public.contracts (
  contract_id uuid NOT NULL DEFAULT gen_random_uuid(),
  contractor_id uuid NOT NULL,
  project_id uuid NOT NULL,
  chapter_id uuid,
  object_description text NOT NULL,
  estimated_total_value numeric NOT NULL,
  status character varying NOT NULL,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  updated_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT contracts_pkey PRIMARY KEY (contract_id),
  CONSTRAINT contracts_contractor_id_fkey FOREIGN KEY (contractor_id) REFERENCES public.users(id),
  CONSTRAINT contracts_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(project_id),
  CONSTRAINT contracts_chapter_id_fkey FOREIGN KEY (chapter_id) REFERENCES public.chapters(chapter_id),
  CONSTRAINT contracts_estimated_total_value_check CHECK (estimated_total_value > 0),
  CONSTRAINT contracts_status_check CHECK (status::text = ANY (ARRAY['active','suspended','pending_closure','closed']))
);

CREATE TABLE public.contract_advances (
  advance_id uuid NOT NULL DEFAULT gen_random_uuid(),
  contract_id uuid NOT NULL,
  execution_value numeric NOT NULL,
  advance_date date NOT NULL,
  supporting_doc_url text,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT contract_advances_pkey PRIMARY KEY (advance_id),
  CONSTRAINT contract_advances_contract_id_fkey FOREIGN KEY (contract_id) REFERENCES public.contracts(contract_id),
  CONSTRAINT contract_advances_execution_value_check CHECK (execution_value >= 0)
);

CREATE TABLE public.commitments (
  commitment_id uuid NOT NULL DEFAULT gen_random_uuid(),
  project_id uuid,
  user_id uuid NOT NULL,
  contract_id uuid,
  commitment_type character varying NOT NULL,
  invoice_format character varying,
  invoice_number character varying,
  commitment_date date NOT NULL,
  gross_value numeric NOT NULL,
  tax_amount numeric NOT NULL DEFAULT 0.00,
  retention_amount numeric NOT NULL DEFAULT 0.00,
  retention_type_rate_id uuid,
  advance_amount numeric NOT NULL DEFAULT 0,
  invoice_total numeric GENERATED ALWAYS AS (gross_value + tax_amount) STORED,
  net_payable numeric GENERATED ALWAYS AS (((gross_value + tax_amount) - retention_amount) - advance_amount) STORED,
  accounting_number character varying,
  supporting_doc_url text,
  notes text,
```

```
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  updated_at timestamp with time zone NOT NULL DEFAULT now(),
  created_by uuid NOT NULL DEFAULT auth.uid(),
  CONSTRAINT commitments_pkey PRIMARY KEY (commitment_id),
  CONSTRAINT commitments_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(project_id),
  CONSTRAINT commitments_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.users(id),
  CONSTRAINT commitments_contract_id_fkey FOREIGN KEY (contract_id) REFERENCES public.contracts(contract_id),
  CONSTRAINT commitments_retention_type_rate_id_fkey FOREIGN KEY (retention_type_rate_id) REFERENCES
public.retention_type_rates(retention_type_rate_id),
  CONSTRAINT commitments_created_by_fkey FOREIGN KEY (created_by) REFERENCES public.profiles(id),
  CONSTRAINT commitments_commitment_type_check CHECK (commitment_type::text = ANY
(ARRAY['invoice','collection_account','contract_settlement'])),
  CONSTRAINT commitments_invoice_format_check CHECK (invoice_format::text = ANY (ARRAY['physical','electronic'])),
  CONSTRAINT commitments_gross_value_check CHECK (gross_value >= 0),
  CONSTRAINT commitments_tax_amount_check CHECK (tax_amount >= 0),
  CONSTRAINT commitments_retention_amount_check CHECK (retention_amount >= 0),
  CONSTRAINT commitments_advance_amount_check CHECK (advance_amount >= 0)
);

CREATE TABLE public.commitment_details (
  detail_id uuid NOT NULL DEFAULT gen_random_uuid(),
  commitment_id uuid NOT NULL,
  activity_id uuid,
  chapter_id uuid,
  description text NOT NULL,
  quantity numeric NOT NULL,
  unit_price numeric NOT NULL,
  total_price numeric GENERATED ALWAYS AS (quantity * unit_price) STORED,
  retention_type_rate_id uuid,
  CONSTRAINT commitment_details_pkey PRIMARY KEY (detail_id),
  CONSTRAINT commitment_details_commitment_id_fkey FOREIGN KEY (commitment_id) REFERENCES public.commitments(commitment_id),
  CONSTRAINT commitment_details_activity_id_fkey FOREIGN KEY (activity_id) REFERENCES public.project_activities(activity_id),
  CONSTRAINT commitment_details_chapter_id_fkey FOREIGN KEY (chapter_id) REFERENCES public.chapters(chapter_id),
  CONSTRAINT commitment_details_retention_type_rate_id_fkey FOREIGN KEY (retention_type_rate_id) REFERENCES
public.retention_type_rates(retention_type_rate_id),
  CONSTRAINT commitment_details_quantity_check CHECK (quantity > 0),
  CONSTRAINT commitment_details_unit_price_check CHECK (unit_price >= 0)
);

-- ---------- Ventas ----------
CREATE TABLE public.sales (
  sale_id uuid NOT NULL DEFAULT gen_random_uuid(),
  customer_id uuid NOT NULL,
  property_id uuid NOT NULL,
  sale_date date NOT NULL,
  separation_amount numeric NOT NULL DEFAULT 0.00,
  total_sale_value numeric NOT NULL,
  down_payment_total numeric NOT NULL,
  credit_amount numeric GENERATED ALWAYS AS (total_sale_value - down_payment_total) STORED,
  sale_status character varying NOT NULL,
  sale_folder_url text,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  updated_at timestamp with time zone NOT NULL DEFAULT now(),
  created_by uuid NOT NULL DEFAULT auth.uid(),
  CONSTRAINT sales_pkey PRIMARY KEY (sale_id),
  CONSTRAINT sales_customer_id_fkey FOREIGN KEY (customer_id) REFERENCES public.users(id),
  CONSTRAINT sales_property_id_fkey FOREIGN KEY (property_id) REFERENCES public.properties(property_id),
  CONSTRAINT sales_created_by_fkey FOREIGN KEY (created_by) REFERENCES public.profiles(id),
  CONSTRAINT sales_sale_status_check CHECK (sale_status::text = ANY (ARRAY['active','completed','canceled'])),
  CONSTRAINT sales_separation_amount_check CHECK (separation_amount >= 0),
  CONSTRAINT sales_total_sale_value_check CHECK (total_sale_value > 0)
);

CREATE TABLE public.payment_plans (
  plan_id uuid NOT NULL DEFAULT gen_random_uuid(),
  sale_id uuid NOT NULL,
  installment_number integer NOT NULL,
  due_date date NOT NULL,
  expected_amount numeric NOT NULL,
  installment_type character varying NOT NULL DEFAULT 'down_payment_installment',
  CONSTRAINT payment_plans_pkey PRIMARY KEY (plan_id),
  CONSTRAINT payment_plans_sale_id_fkey FOREIGN KEY (sale_id) REFERENCES public.sales(sale_id),
  CONSTRAINT payment_plans_installment_number_check CHECK (installment_number > 0),
  CONSTRAINT payment_plans_expected_amount_check CHECK (expected_amount > 0),
  CONSTRAINT payment_plans_installment_type_check CHECK (installment_type::text = ANY
(ARRAY['down_payment_installment','mortgage_credit','subsidy','extra']))
);

-- ---------- Tesoreria ----------
CREATE TABLE public.company_accounts (
  account_id uuid NOT NULL DEFAULT gen_random_uuid(),
```

```
  account_name character varying NOT NULL,
  account_kind character varying NOT NULL,
  bank_id uuid,
  account_type character varying,
  account_number character varying,
  is_active boolean NOT NULL DEFAULT true,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  updated_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT company_accounts_pkey PRIMARY KEY (account_id),
  CONSTRAINT company_accounts_account_name_key UNIQUE (account_name),
  CONSTRAINT company_accounts_bank_id_fkey FOREIGN KEY (bank_id) REFERENCES public.banks(id),
  CONSTRAINT company_accounts_account_kind_check CHECK (account_kind::text = ANY
(ARRAY['bank','petty_cash','cash','investment'])),
  CONSTRAINT company_accounts_account_type_check CHECK (account_type::text = ANY (ARRAY['savings','checking','digital_wallet']))
);

CREATE TABLE public.treasury_movements (
  movement_id uuid NOT NULL DEFAULT gen_random_uuid(),
  movement_type character varying NOT NULL,
  sub_type character varying NOT NULL,
  movement_date date NOT NULL,
  amount numeric NOT NULL,
  project_id uuid,
  user_id uuid,
  commitment_id uuid,
  sale_id uuid,
  contract_id uuid,
  payment_method_id uuid,
  source_account_id uuid,
  dest_account_id uuid,
  dest_bank_account_id uuid,
  supporting_doc_url text,
  notes text,
  created_by uuid NOT NULL,
  created_at timestamp with time zone NOT NULL DEFAULT now(),
  CONSTRAINT treasury_movements_pkey PRIMARY KEY (movement_id),
  CONSTRAINT treasury_movements_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(project_id),
  CONSTRAINT treasury_movements_user_id_fkey FOREIGN KEY (user_id) REFERENCES public.users(id),
  CONSTRAINT treasury_movements_commitment_id_fkey FOREIGN KEY (commitment_id) REFERENCES public.commitments(commitment_id),
  CONSTRAINT treasury_movements_sale_id_fkey FOREIGN KEY (sale_id) REFERENCES public.sales(sale_id),
  CONSTRAINT treasury_movements_contract_id_fkey FOREIGN KEY (contract_id) REFERENCES public.contracts(contract_id),
  CONSTRAINT treasury_movements_payment_method_id_fkey FOREIGN KEY (payment_method_id) REFERENCES public.payment_methods(id),
  CONSTRAINT treasury_movements_source_account_id_fkey FOREIGN KEY (source_account_id) REFERENCES
public.company_accounts(account_id),
  CONSTRAINT treasury_movements_dest_account_id_fkey FOREIGN KEY (dest_account_id) REFERENCES public.company_accounts(account_id),
  CONSTRAINT treasury_movements_dest_bank_account_id_fkey FOREIGN KEY (dest_bank_account_id) REFERENCES public.bank_accounts(id),
  CONSTRAINT treasury_movements_created_by_fkey FOREIGN KEY (created_by) REFERENCES public.profiles(id),
  CONSTRAINT treasury_movements_amount_check CHECK (amount > 0),
  CONSTRAINT treasury_movements_movement_type_check CHECK (movement_type::text = ANY
(ARRAY['income','expense','transfer','investment'])),
  CONSTRAINT treasury_movements_sub_type_check CHECK (sub_type::text = ANY
(ARRAY['sale_payment','sale_refund','contract_advance','commitment_payment','petty_cash_funding','petty_cash_expense','internal_tr
ansfer','cdt','loan']))
);

CREATE TABLE public.commitment_applications (
  application_id uuid NOT NULL DEFAULT gen_random_uuid(),
  movement_id uuid NOT NULL,
  commitment_id uuid NOT NULL,
  applied_amount numeric NOT NULL,
  CONSTRAINT commitment_applications_pkey PRIMARY KEY (application_id),
  CONSTRAINT commitment_applications_movement_id_fkey FOREIGN KEY (movement_id) REFERENCES public.treasury_movements(movement_id),
  CONSTRAINT commitment_applications_commitment_id_fkey FOREIGN KEY (commitment_id) REFERENCES public.commitments(commitment_id),
  CONSTRAINT commitment_applications_applied_amount_check CHECK (applied_amount > 0)
);

CREATE TABLE public.payment_applications (
  application_id uuid NOT NULL DEFAULT gen_random_uuid(),
  movement_id uuid NOT NULL,
  plan_id uuid NOT NULL,
  applied_amount numeric NOT NULL,
  CONSTRAINT payment_applications_pkey PRIMARY KEY (application_id),
  CONSTRAINT payment_applications_movement_id_fkey FOREIGN KEY (movement_id) REFERENCES public.treasury_movements(movement_id),
  CONSTRAINT payment_applications_plan_id_fkey FOREIGN KEY (plan_id) REFERENCES public.payment_plans(plan_id),
  CONSTRAINT payment_applications_applied_amount_check CHECK (applied_amount > 0)
);

-- ============================================================================
-- Funciones y triggers (5 + 5) tal como existen en la copia
-- ============================================================================
CREATE OR REPLACE FUNCTION public.fn_set_updated_at() RETURNS trigger LANGUAGE plpgsql AS $$
```

```
BEGIN NEW.updated_at = now(); RETURN NEW; END; $$;

CREATE OR REPLACE FUNCTION public.fn_property_block_must_be_housing() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path TO 'public', 'pg_temp' AS $$
DECLARE v_block_kind VARCHAR(20);
BEGIN
  SELECT block_kind INTO v_block_kind FROM blocks WHERE block_id = NEW.block_id AND project_id = NEW.project_id;
  IF v_block_kind IS DISTINCT FROM 'housing' THEN
    RAISE EXCEPTION 'Property % can only be assigned to a block of type housing (block received: %)', NEW.property_id,
v_block_kind;
  END IF;
  RETURN NEW;
END; $$;

CREATE OR REPLACE FUNCTION public.fn_contract_contractor_must_be_contractor() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path TO 'public', 'pg_temp' AS $$
DECLARE v_user_type VARCHAR(20);
BEGIN
  SELECT user_type INTO v_user_type FROM users WHERE id = NEW.contractor_id;
  IF v_user_type IS DISTINCT FROM 'contractor' THEN
    RAISE EXCEPTION 'contractor_id % must have user_type = contractor (has: %)', NEW.contractor_id, v_user_type;
  END IF;
  RETURN NEW;
END; $$;

CREATE OR REPLACE FUNCTION public.fn_sync_property_status_with_sale() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path TO 'public', 'pg_temp' AS $$
BEGIN
  IF NEW.sale_status IN ('active','completed') THEN
    UPDATE properties SET status = 'sold' WHERE property_id = NEW.property_id;
  ELSIF NEW.sale_status = 'canceled' THEN
    UPDATE properties SET status = 'available' WHERE property_id = NEW.property_id;
  END IF;
  RETURN NEW;
END; $$;

CREATE OR REPLACE FUNCTION public.fn_apply_sale_payment_fifo() RETURNS trigger
LANGUAGE plpgsql SET search_path TO 'public', 'pg_temp' AS $$
DECLARE
  v_remaining DECIMAL(15,2); v_plan RECORD; v_already_applied DECIMAL(15,2); v_to_apply DECIMAL(15,2);
BEGIN
  IF NEW.movement_type <> 'income' OR NEW.sub_type <> 'sale_payment' OR NEW.sale_id IS NULL THEN
    RETURN NEW;
  END IF;
  v_remaining := NEW.amount;
  FOR v_plan IN SELECT plan_id, expected_amount FROM payment_plans
                WHERE sale_id = NEW.sale_id ORDER BY installment_number FOR UPDATE
  LOOP
    EXIT WHEN v_remaining <= 0;
    SELECT COALESCE(SUM(applied_amount), 0) INTO v_already_applied
      FROM payment_applications WHERE plan_id = v_plan.plan_id;
    IF v_already_applied >= v_plan.expected_amount THEN CONTINUE; END IF;
    v_to_apply := LEAST(v_remaining, v_plan.expected_amount - v_already_applied);
    INSERT INTO payment_applications (movement_id, plan_id, applied_amount)
      VALUES (NEW.movement_id, v_plan.plan_id, v_to_apply);
    v_remaining := v_remaining - v_to_apply;
  END LOOP;
  RETURN NEW;
END; $$;

CREATE TRIGGER trg_projects_set_updated_at BEFORE UPDATE ON public.projects
  FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();
CREATE TRIGGER trg_property_block_must_be_housing BEFORE INSERT OR UPDATE OF block_id, project_id ON public.properties
  FOR EACH ROW EXECUTE FUNCTION fn_property_block_must_be_housing();
CREATE TRIGGER trg_contract_contractor_must_be_contractor BEFORE INSERT OR UPDATE OF contractor_id ON public.contracts
  FOR EACH ROW EXECUTE FUNCTION fn_contract_contractor_must_be_contractor();
CREATE TRIGGER trg_sync_property_status_with_sale AFTER INSERT OR UPDATE OF sale_status ON public.sales
  FOR EACH ROW EXECUTE FUNCTION fn_sync_property_status_with_sale();
CREATE TRIGGER trg_apply_sale_payment_fifo AFTER INSERT ON public.treasury_movements
  FOR EACH ROW EXECUTE FUNCTION fn_apply_sale_payment_fifo();
```

Script DML (carga de datos)

Los datos de prueba son sintéticos. Se entregan como script DML real: datos_completos.sql, el respaldo de datos de la base desplegada (pgAdmin 4, esquema public, solo datos, formato INSERT; 7.053 sentencias para las 29 tablas, cuyos conteos coinciden con la base). La carga

complementaria del 09/11/2026 se hizo con el script completar_datos.sql (se ejecutó primero con v_dry = true y luego con v_dry = false), que también se entrega como archivo adjunto. Extracto del respaldo, orden de carga y verificación, y a continuación el script de la carga complementaria:

```
-- Extracto de datos_completos.sql (respaldo de las 29 tablas, 7.053 sentencias INSERT; archivo adjunto).
-- Se carga despues de crear el esquema (esquema_copia.sql). Con session_replication_role = replica
-- no se disparan los triggers (trg_apply_sale_payment_fifo duplicaria payment_applications, que ya viene
-- en el respaldo) ni se validan las llaves foraneas durante la carga; al terminar se restablece.
SET session_replication_role = replica;

INSERT INTO public.banks VALUES ('af29a1a4-5e2a-4230-8ac5-180e942749c4', 'Bancolombia', true, '2026-11-07 21:48:47.02628+00',
'2026-11-07 21:48:47.02628+00');
INSERT INTO public.banks VALUES ('d45d5929-4bfb-4424-abae-cd0a880d8f6b', 'Davivienda', true, '2026-11-07 21:48:47.02628+00',
'2026-11-07 21:48:47.02628+00');
INSERT INTO public.projects VALUES ('0e97bd35-2c1e-4ffc-ad64-e193498912e6', 'Proyecto 0001 Torres de El Bosque', 'Ocaña,
Colombia', 'active', '2023-01-02', '2023-10-30', 12, '2026-11-07 21:49:13.767733+00', '2026-11-07 21:49:13.767733+00');
INSERT INTO public.projects VALUES ('174b429a-cce5-4ddf-a3eb-49062a1ddf62', 'Proyecto 0002 Urbanización Altos del Valle',
'Bucaramanga, Colombia', 'active', '2023-01-03', '2023-11-01', 19, '2026-11-07 21:49:13.767733+00', '2026-11-07
21:49:13.767733+00');
-- ... (7.049 sentencias INSERT mas, una por fila)

RESET session_replication_role;

-- Orden logico de las tablas (respeta las llaves foraneas si se cargan sin el modo replica):
-- 1) banks, payment_methods, chapters, units, retention_types, retention_type_rates, labor_types
-- 2) profiles, users, bank_accounts
-- 3) projects, blocks, activity_catalog, properties, project_activities
-- 4) contracts, contract_advances, commitments, commitment_details, sales, payment_plans
-- 5) company_accounts, treasury_movements, commitment_applications, payment_applications
-- 6) work_progress, labor_performance, labor_performance_contractors, labor_performance_crew
-- Verificacion de la carga (conteo de filas):
SELECT relname AS tabla, n_live_tup AS filas FROM pg_stat_user_tables ORDER BY relname;
-- ============================================================================
-- Hub TuCasaConstrucciones - Carga complementaria de datos sinteticos (09/11/2026)
-- Corrige los 3 hallazgos de la revision de calidad (Anexo 1, E4):
--   1) 470 compromisos sin items         -> commitment_details (+ commitment_applications vacia)
--   2) 250 proyectos sin inmuebles       -> properties
--   3) 20 proveedores/contratistas sin cuenta bancaria -> bank_accounts
-- No toca las cantidades pedidas por el profesor (projects, project_activities,
-- commitments, treasury_movements y sales quedan igual).
-- Los datos se generan de forma DETERMINISTA (hashtext) a partir de los datos existentes.
-- v_dry = true  -> prueba: valida y REVIERTE todo (termina en una excepcion informativa).
-- v_dry = false -> confirma los cambios.
-- ============================================================================
DO $$
DECLARE
  v_dry boolean := false;
  n_items int;
  n_apps int;
  n_props int;
  n_accts int;
  bad int;
BEGIN
  -- 1) Items de los compromisos que no tienen (1 a 3 por compromiso; la suma de los items = valor bruto)
  WITH c AS (
    SELECT commitment_id, project_id, gross_value,
           1 + abs(hashtext(commitment_id::text)) % 3 AS n
    FROM commitments c
    WHERE NOT EXISTS (SELECT 1 FROM commitment_details d WHERE d.commitment_id = c.commitment_id)
  ),
  act AS (
    SELECT activity_id, project_id, chapter_id,
           row_number() OVER (PARTITION BY project_id ORDER BY activity_id) AS rn,
           count(*) OVER (PARTITION BY project_id) AS k
    FROM project_activities
  ),
  it AS (
    SELECT c.commitment_id, c.project_id, c.gross_value, c.n, g.i,
           1 + abs(hashtext(c.commitment_id::text || '-' || g.i)) % 5 AS w
    FROM c
    CROSS JOIN LATERAL generate_series(1, c.n) AS g(i)
  ),
  sized AS (
    SELECT *, sum(w) OVER (PARTITION BY commitment_id) AS sw FROM it
  ),
  part AS (
    SELECT *, CASE WHEN i < n THEN floor(gross_value * w / sw / 10000) * 10000 END AS p FROM sized
```

```
),
fin AS (
  SELECT *, CASE WHEN i < n THEN p
                 ELSE gross_value - COALESCE(sum(p) OVER (PARTITION BY commitment_id), 0) END AS amount
  FROM part
),
pick AS (
  SELECT f.*, a.activity_id, a.chapter_id
  FROM fin f
  JOIN act a ON a.project_id = f.project_id AND a.rn = ((f.i - 1) % a.k) + 1
)
INSERT INTO commitment_details (commitment_id, activity_id, chapter_id, description, quantity, unit_price)
SELECT pk.commitment_id, pk.activity_id, pk.chapter_id,
       (SELECT ch.chapter_name FROM chapters ch WHERE ch.chapter_id = pk.chapter_id) || ': ' ||
       (ARRAY['suministro de materiales', 'mano de obra', 'alquiler de equipo', 'transporte',
              'servicios tecnicos', 'obra complementaria'])[1 + abs(hashtext(pk.commitment_id::text || '/' || pk.i)) % 6],
       q.qty,
       pk.amount / q.qty
FROM pick pk
CROSS JOIN LATERAL (
  SELECT CASE WHEN pk.amount % (ARRAY[1, 2, 4, 5, 10])[1 + abs(hashtext(pk.commitment_id::text || '#' || pk.i)) % 5] = 0
              THEN (ARRAY[1, 2, 4, 5, 10])[1 + abs(hashtext(pk.commitment_id::text || '#' || pk.i)) % 5]
              ELSE 1 END AS qty
) q;
GET DIAGNOSTICS n_items = ROW_COUNT;

-- 1b) Aplicacion de cada pago de compromiso a su compromiso (commitment_applications estaba vacia)
INSERT INTO commitment_applications (movement_id, commitment_id, applied_amount)
SELECT tm.movement_id, tm.commitment_id, tm.amount
FROM treasury_movements tm
WHERE tm.sub_type = 'commitment_payment'
  AND tm.commitment_id IS NOT NULL
  AND NOT EXISTS (SELECT 1 FROM commitment_applications ca WHERE ca.movement_id = tm.movement_id);
GET DIAGNOSTICS n_apps = ROW_COUNT;

-- 2) Un inmueble para cada proyecto que no tiene (en su bloque de viviendas, codigo P-####-101)
INSERT INTO properties (project_id, block_id, property_code, property_type, area_sqm, total_price, status)
SELECT p.project_id, b.block_id,
       'P-' || lpad(substring(p.project_name from 'Proyecto ([0-9]+)'), 4, '0') || '-101',
       (ARRAY['Apartamento', 'Casa', 'Local comercial'])[1 + abs(hashtext(p.project_id::text)) % 3],
       a.area,
       a.area * (3000000 + 250000 * (abs(hashtext(p.project_id::text || 'precio')) % 10)),
       'available'
FROM projects p
JOIN blocks b ON b.project_id = p.project_id AND b.block_kind = 'housing'
CROSS JOIN LATERAL (SELECT 40 + abs(hashtext(p.project_id::text || 'area')) % 81 AS area) a
WHERE NOT EXISTS (SELECT 1 FROM properties x WHERE x.project_id = p.project_id);
GET DIAGNOSTICS n_props = ROW_COUNT;

-- 3) Cuenta bancaria principal para cada proveedor o contratista que no tiene
INSERT INTO bank_accounts (user_id, bank_id, account_type, account_number, is_primary)
SELECT u.id,
       (SELECT bk.id FROM banks bk ORDER BY bk.id OFFSET (abs(hashtext(u.id::text)) % 15) LIMIT 1),
       (ARRAY['savings', 'checking', 'digital_wallet'])[1 + abs(hashtext(u.id::text || 'tipo')) % 3],
       '70' || lpad((abs(hashtext(u.id::text || 'cuenta')) % 100000000)::text, 8, '0'),
       true
FROM users u
WHERE u.user_type IN ('provider', 'contractor')
  AND NOT EXISTS (SELECT 1 FROM bank_accounts b WHERE b.user_id = u.id);
GET DIAGNOSTICS n_accts = ROW_COUNT;

-- Validaciones: si alguna falla, la excepcion revierte TODO
SELECT count(*) INTO bad FROM commitments c
WHERE NOT EXISTS (SELECT 1 FROM commitment_details d WHERE d.commitment_id = c.commitment_id);
IF bad > 0 THEN RAISE EXCEPTION 'Quedan % compromisos sin items', bad; END IF;

SELECT count(*) INTO bad FROM (
  SELECT d.commitment_id FROM commitment_details d JOIN commitments c ON c.commitment_id = d.commitment_id
  GROUP BY d.commitment_id, c.gross_value HAVING sum(d.total_price) <> c.gross_value) x;
IF bad > 0 THEN RAISE EXCEPTION '% compromisos con suma de items distinta del valor bruto', bad; END IF;

SELECT count(*) INTO bad FROM (
  SELECT ca.commitment_id FROM commitment_applications ca JOIN commitments c ON c.commitment_id = ca.commitment_id
  GROUP BY ca.commitment_id, c.net_payable HAVING sum(ca.applied_amount) > c.net_payable) x;
IF bad > 0 THEN RAISE EXCEPTION '% compromisos pagados por encima del neto a pagar', bad; END IF;

SELECT count(*) INTO bad FROM projects p WHERE NOT EXISTS (SELECT 1 FROM properties x WHERE x.project_id = p.project_id);
IF bad > 0 THEN RAISE EXCEPTION 'Quedan % proyectos sin inmuebles', bad; END IF;

SELECT count(*) INTO bad FROM (SELECT property_code FROM properties GROUP BY property_code HAVING count(*) > 1) x;
IF bad > 0 THEN RAISE EXCEPTION '% codigos de inmueble repetidos', bad; END IF;
```

```
  SELECT count(*) INTO bad FROM users u
  WHERE u.user_type IN ('provider', 'contractor') AND NOT EXISTS (SELECT 1 FROM bank_accounts b WHERE b.user_id = u.id);
  IF bad > 0 THEN RAISE EXCEPTION 'Quedan % proveedores o contratistas sin cuenta bancaria', bad; END IF;

  SELECT count(*) INTO bad FROM (SELECT account_number FROM bank_accounts GROUP BY account_number HAVING count(*) > 1) x;
  IF bad > 0 THEN RAISE EXCEPTION '% numeros de cuenta repetidos', bad; END IF;

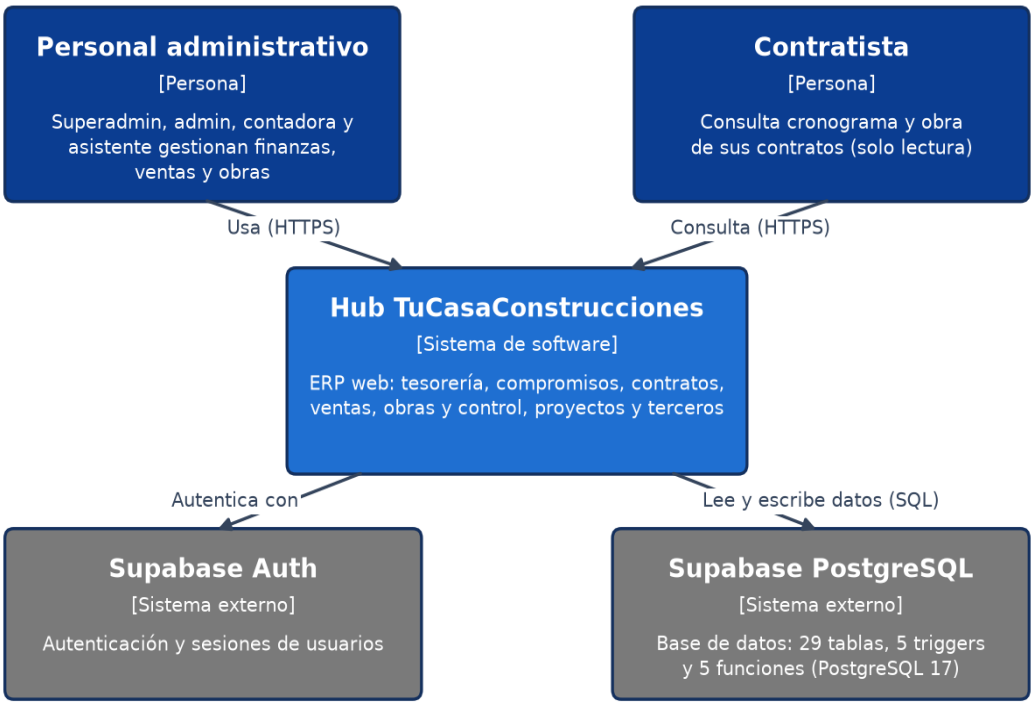
  IF v_dry THEN
    RAISE EXCEPTION 'PRUEBA OK (se revirtio todo): items nuevos=%, aplicaciones=%, inmuebles=%, cuentas=%',
      n_items, n_apps, n_props, n_accts;
  END IF;
  RAISE NOTICE 'CARGA CONFIRMADA: items nuevos=%, aplicaciones=%, inmuebles=%, cuentas=%',
    n_items, n_apps, n_props, n_accts;
END $$;
```

4.3 Modelo C4

Nivel 1: Diagrama contexto sistema

El sistema Hub TuCasaConstrucciones lo usa el personal administrativo (superadmin, admin, contadora y asistente) y los contratistas, que solo consultan. Depende de Supabase para la autenticación (Supabase Auth) y para el almacenamiento de los datos (PostgreSQL con 29 tablas, 5 triggers y 5 funciones).

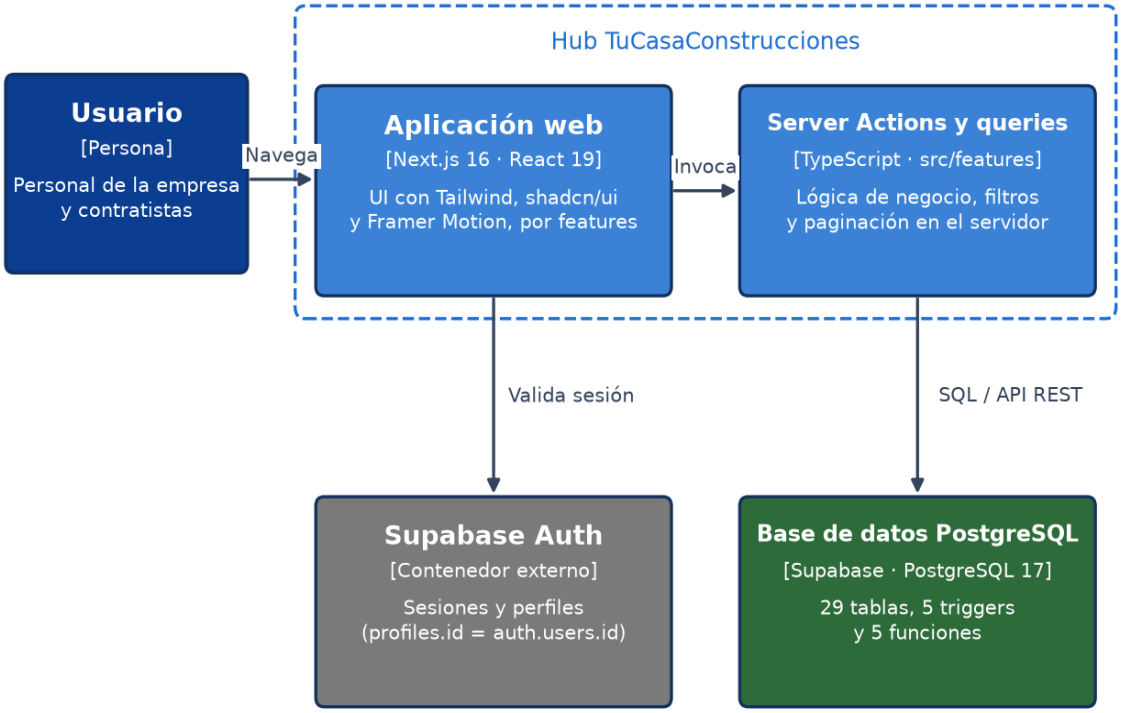
C4 - Nivel 1: Diagrama de contexto del sistema



Nivel 2: Diagrama contenedor

El sistema se compone de una aplicación web Next.js (App Router) organizada por features, Server Actions y consultas que ejecutan la lógica, los filtros y la paginación en el servidor, Supabase Auth para las sesiones y los perfiles, y la base de datos PostgreSQL de Supabase (tablas, triggers y funciones).

C4 - Nivel 2: Diagrama de contenedores



4.4 Desarrollo web

La aplicación se desarrolló con Next.js 16 (App Router), React 19 y TypeScript, con Tailwind CSS 4, shadcn/ui, Radix UI y Framer Motion para la interfaz, Recharts para las gráficas y Supabase (@supabase/ssr y supabase-js) para autenticación y datos. El código se organiza en src/features/[módulo] con sus componentes, actions.ts (mutaciones) y queries.ts (consultas); los textos de la interfaz se extraen a archivos de idioma (es.json y en.json) y las tablas y listados pesados filtran, buscan y paginan en el servidor.

Módulos implementados: Proyectos, Terceros, Compromisos, Contratos, Tesorería (financiero), Ventas, Presupuesto, Cronograma, Avance de obra, Rendimiento de personal, Catálogos y Administración de accesos.

Alcance respecto a esta entrega: el modelo de datos entregado y evaluado es el de la base desplegada en Supabase (29 tablas, 5 triggers y 5 funciones). Los objetos que la aplicación productiva agrega por encima de ese modelo (vistas de saldos v_*, la función save_commitment y las políticas RLS por rol) no forman parte de esta copia y se listan en las recomendaciones.

4.5 Repositorio Github

https://github.com/CASTILLA057/tucasaconstrucciones-bd

5. Transición

5.1 Pruebas de aceptación de usuario (UAT)

Las pruebas de aceptación de usuario (UAT) validan que la base de datos desplegada cumple los requisitos y las reglas del negocio. Se diseñaron contra el esquema real de la copia (29 tablas, 5 triggers y 5 funciones); las consultas están en el archivo uat_copia.sql, que se ejecuta en pgAdmin 4 o en el SQL Editor de Supabase y no deja datos en la base (las pruebas de escritura terminan en una excepción que revierte todo).

Informe UAT

Fecha del Informe: 08/11/2026
Responsable: Santiago Castilla Coronel
Versión del Sistema: esquema de la copia (29 tablas, 5 triggers y 5 funciones; PostgreSQL 17.11)
Entorno de Prueba: Supabase, proyecto base_de_datos (copia), región us-east-1, plan Free
Resumen: 28 casos; 26 aprobados y 2 con hallazgo de seguridad (SEC-01 y SEC-02).

Estrategia de Pruebas

Tipos de Pruebas

Tipo de Prueba	Descripción
Pruebas de Integridad de Datos	Verifica que los datos insertados, actualizados y eliminados cumplen con las restricciones de integridad (llaves, CHECK, UNIQUE, columnas calculadas) y que los cinco triggers automatizan las reglas de procedimiento.
Pruebas de Consultas y Reportes	Valida que las consultas SQL con JOIN y funciones agregadas devuelven los resultados esperados sobre los datos poblados.
Pruebas de Concurrencia	Evalúa el comportamiento de la base de datos con dos sesiones accediendo simultáneamente a la misma fila.
Pruebas de Seguridad	Verifica la autenticación, autorización y restricciones de acceso a los datos (RLS, privilegios y SQL injection).
Pruebas de Rendimiento	Mide el tiempo de ejecución de consultas y la eficiencia de los índices, incluso con un millón de filas.
Pruebas de Recuperación ante Fallos	Simula fallos y verifica la recuperación de datos.

Casos de Prueba

Pruebas de Integridad de Datos y de Lógica en la Base de Datos (triggers)

ID	Caso de Prueba	Consulta SQL	Resultado Esperado	Resultado Obtenido	Estado (Aprobado / Fallo / Pendiente)
INT-01	Llave foránea: bloque con proyecto inexistente	INSERT INTO blocks (project_id, block_name, block_kind) VALUES ('00000000-0000-0000-0000-000000000000', 'X', 'housing');	Error 23503 (blocks_project_id_fkey)	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Error 23503: viola la llave foránea blocks_project_id_fkey.	Aprobado
INT-02	UNIQUE: nombre de banco	INSERT INTO banks (name) SELECT name FROM banks LIMIT 1;	Error 23505 (banks_name_key)	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Error 23505: viola la restricción única	Aprobado

	repetido			banks_name_key.	
INT-03	CHECK: área de un inmueble negativa	INSERT INTO properties (project_id, block_id, property_code, property_type, area_sqm, total_price) SELECT project_id, block_id, 'T-001', 'Casa', -5, 1 FROM blocks WHERE block_kind = 'housing' LIMIT 1;	Error 23514 (properties_area_sqm_check)	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Error 23514: viola la restricción CHECK properties_area_sqm_check.	Aprobado
INT-04	CHECK: monto de tesorería igual a 0	INSERT INTO treasury_movements (movement_type, sub_type, movement_date, amount, created_by) SELECT 'income', 'sale_payment', current_date, 0, id FROM profiles LIMIT 1;	Error 23514 (treasury_movements_amount_check)	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Error 23514: viola la restricción CHECK treasury_movements_amount_check.	Aprobado
INT-05	CHECK de dominio: estado de proyecto inválido	INSERT INTO projects (project_name, project_location, status) VALUES ('Prueba', 'Ocaña', 'invalid');	Error 23514 (projects_status_check)	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Error 23514: viola la restricción CHECK projects_status_check.	Aprobado
INT-06	Columna calculada no editable	INSERT INTO sales (customer_id, property_id, sale_date, total_sale_value, down_payment_total, sale_status, credit_amount) VALUES (gen_random_uuid(), gen_random_uuid(), current_date, 100, 20, 'active', 50);	Error: no se puede insertar un valor en la columna generada credit_amount	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Error 428C9: no se puede insertar un valor distinto de DEFAULT en la columna credit_amount. Además, una venta de 100.000.000 con cuota inicial de 20.000.000 calculó credit_amount = 80.000.000.	Aprobado
INT-07	Valores calculados consistentes	SELECT count(*) FROM commitments WHERE net_payable <> gross_value + tax_amount - retention_amount - advance_amount;	0 filas con diferencia	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. 0 compromisos con diferencia.	Aprobado
INT-08	Documentos de tercero duplicados (la base no tiene UNIQUE)	SELECT document_type, document_number, count(*) FROM users GROUP BY 1, 2 HAVING count(*) > 1;	Sin filas (la aplicación evita duplicados; la base no lo impide)	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. 0 grupos de documento duplicados (la base no lo impide con una restricción; los datos no tienen duplicados).	Aprobado
TRG-01	Inmueble solo en bloque housing	INSERT INTO properties (...) SELECT ... FROM blocks WHERE block_kind = 'common_areas' LIMIT 1;	Excepción: el inmueble solo puede asignarse a un bloque de tipo housing	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Excepción P0001: el inmueble solo puede asignarse a un bloque de tipo housing (bloque recibido: common_areas).	Aprobado
TRG-02	Contratista de tipo contractor	INSERT INTO contracts (...) con un tercero de tipo client como contratista;	Excepción: contractor_id debe tener user_type = contractor	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Excepción P0001: contractor_id debe tener user_type = contractor (tiene: client).	Aprobado
TRG-03	La venta sincroniza el estado del inmueble	Insertar una venta activa y luego cancelarla; consultar properties.status en cada paso.	El inmueble pasa a sold y vuelve a available	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Tras insertar la venta activa el inmueble quedó en sold; tras cancelarla volvió a available.	Aprobado
TRG-04	Aplicación FIFO de un pago	Con una venta de dos cuotas de 10.000.000, insertar un movimiento income / sale_payment de 15.000.000.	payment_applications: 10.000.000 a la cuota 1 y 5.000.000 a la cuota 2	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. payment_applications: cuota 1 recibió 10.000.000 y cuota 2 recibió 5.000.000.	Aprobado
TRG-05	updated_at de proyectos	UPDATE projects SET progress = progress WHERE project_id = (SELECT project_id FROM projects LIMIT 1);	updated_at cambia al valor de now()	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. updated_at cambió de 2026-11-07 21:49:13 a 2026-11-09 04:12:57 (UTC).	Aprobado

Observaciones: Evaluación general: 13 de 13 casos aprobados (INT-01 a INT-08 y TRG-01 a TRG-05). Problemas críticos identificados: ninguno; la base no impide documentos de tercero duplicados ni dos ventas vigentes sobre un inmueble, aunque los datos actuales no los tienen. Recomendaciones antes del despliegue: agregar UNIQUE (document_type, document_number) en users y un índice único parcial sobre sales (property_id). Las pruebas de escritura se ejecutaron dentro de un bloque que termina en una excepción y se verificó que no quedó ningún dato de prueba.

Pruebas de Consultas y Reportes

ID	Caso de Prueba	Consulta SQL	Resultado Esperado	Resultado Obtenido	Estado (Aprobado / Fallo / Pendiente)
QRY-01	Clientes con más de una compra	SELECT customer_id, COUNT(*) FROM sales GROUP BY customer_id HAVING COUNT(*) > 1;	Lista de clientes con 2 o más ventas	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. 20 clientes con más de una compra.	Aprobado
QRY-02	Ventas por mes (JOIN y agregación)	SELECT date_trunc('month', sale_date) AS mes, SUM(total_sale_value) FROM sales WHERE sale_status <> 'canceled' GROUP BY 1 ORDER BY 1;	Total vendido por mes	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Ventas no canceladas en 7 meses distintos; total vendido 61.617.000.000.	Aprobado
QRY-03	Facturas con saldo pendiente	SELECT c.invoice_number, u.name, c.net_payable, COALESCE(SUM(tm.amount), 0) AS pagado FROM commitments c JOIN users u ON u.id = c.user_id LEFT JOIN treasury_movements tm ON tm.commitment_id = c.commitment_id AND tm.sub_type = 'commitment_payment' GROUP BY c.commitment_id, u.name HAVING c.net_payable - COALESCE(SUM(tm.amount), 0) > 0;	Facturas cuyo neto a pagar supera lo pagado	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. 500 facturas con saldo pendiente: ninguna está pagada por completo, porque solo hay 150 movimientos de pago de compromisos para 500 compromisos.	Aprobado
QRY-04	Total pagado por acreedor (10 mayores)	SELECT u.name, u.user_type, COUNT(*) AS pagos, SUM(tm.amount) AS total FROM treasury_movements tm JOIN commitments c ON c.commitment_id = tm.commitment_id JOIN users u ON u.id = c.user_id WHERE tm.sub_type = 'commitment_payment' GROUP BY u.id, u.name, u.user_type ORDER BY total DESC LIMIT 10;	Los 10 acreedores con mayor total pagado	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. 20 acreedores con pagos; el mayor es Juan Ruiz (provider): 8 pagos por 49.394.500.	Aprobado
QRY-05	Cantidad de filas de las tablas principales	SELECT relname, n_live_tup FROM pg_stat_user_tables WHERE relname IN ('projects', 'project_activities', 'commitments', 'treasury_movements', 'sales');	500, 1000, 500, 500 y 200 filas respectivamente	Conteo exacto con count(*) el 08/11/2026: projects 500; project_activities 1000; commitments 500; treasury_movements 500; sales 200.	Aprobado

Observaciones: Evaluación general: 5 de 5 casos aprobados. Problemas críticos identificados: ninguno; QRY-03 muestra que ninguna de las 500 facturas está pagada por completo (solo hay 150 pagos de compromisos), una característica de los datos sintéticos. Recomendaciones antes del despliegue: crear vistas para los saldos de facturas y poblar commitment_applications.

Pruebas de Concurrencia

ID	Caso de Prueba	Consulta SQL	Resultado Esperado	Resultado Obtenido	Estado (Aprobado / Fallo / Pendiente)
CON-01	Dos usuarios actualizan el mismo registro simultáneamente	Sesión A: UPDATE projects SET progress = 99 sobre una fila y la mantiene abierta 120 s (pg_sleep). Sesión B: UPDATE sobre la misma fila.	La sesión B espera hasta que A termine (bloqueo de fila) y no se pierde ninguna actualización	Ejecutado el 08/11/2026 en dos sesiones del SQL Editor. La sesión B esperó 100,1 s hasta que A terminó (rollback) y solo entonces actualizó la fila.	Aprobado
CON-02	Lectura mientras se realiza una escritura	Con la sesión A escribiendo sin confirmar (verificado en pg_stat_activity), la sesión B ejecuta SELECT progress sobre la misma fila.	La lectura no se bloquea y devuelve el valor confirmado (aislamiento READ COMMITTED)	Ejecutado el 08/11/2026. Con la sesión A activa, la sesión B leyó progress = 12 (el valor confirmado, no el 99 sin	Aprobado

				confirmar) en 7 ms.	

Observaciones: Evaluación general: 2 de 2 casos aprobados. Problemas críticos identificados: ninguno; PostgreSQL bloquea la fila escrita (la otra escritura esperó hasta que terminó la transacción) y las lecturas no se bloquean. Recomendaciones antes del despliegue: usar transacciones cortas y definir lock_timeout y statement_timeout en la aplicación para evitar esperas largas.

Pruebas de Seguridad

ID	Caso de Prueba	Consulta SQL	Resultado Esperado	Resultado Obtenido	Estado (Aprobado / Fallo / Pendiente)
SEC-01	RLS por rol	SELECT count(*) FILTER (WHERE relrowsecurity) AS tablas_con_rls, (SELECT count(*) FROM pg_policies WHERE schemaname = 'public') AS politicas FROM pg_class WHERE relnamespace = 'public'::regnamespace AND relkind = 'r';	RLS activo en las tablas, con políticas por rol	0 tablas con RLS y 0 políticas (consulta ejecutada el 08/11/2026 en el SQL Editor de Supabase).	Fallo (hallazgo)
SEC-02	Privilegios del rol anónimo	SELECT has_table_privilege('anon', 'public.users', 'SELECT');	false (el rol anónimo no debe leer terceros)	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. true: el rol anon tiene permiso SELECT sobre public.users.	Fallo (hallazgo)
SEC-03	SQL Injection	Enviar ' OR 1=1 -- en un campo de entrada y comparar una consulta parametrizada (WHERE email = $1) con una concatenada.	La consulta parametrizada no devuelve filas	Ejecutado el 08/11/2026. Consulta parametrizada: 0 filas. Consulta construida por concatenación: 75 filas (todos los usuarios).	Aprobado (solo con consultas parametrizadas)

Observaciones: Evaluación general: 1 de 3 casos aprobado (SEC-03) y 2 hallazgos (SEC-01 y SEC-02). Problemas críticos identificados: la base no activa RLS y el rol anónimo tiene permiso de lectura sobre las tablas del esquema public; con la API de Supabase esto podría exponer los datos con la clave anónima. Recomendaciones antes del despliegue: activar RLS con políticas por rol, revocar privilegios innecesarios al rol anon y usar siempre consultas parametrizadas.

Pruebas de Rendimiento

ID	Caso de Prueba	Consulta SQL	Resultado Esperado	Resultado Obtenido	Estado (Aprobado / Fallo / Pendiente)
PER-01	Consulta de varias tablas con agregación	EXPLAIN ANALYZE <consulta QRY-03>	Tiempo de ejecución menor a 2 segundos	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Execution Time: 1,415 ms (Seq Scan sobre commitments, users y treasury_movements).	Aprobado
PER-02	Búsqueda por llave foránea sin índice	EXPLAIN ANALYZE SELECT * FROM treasury_movements WHERE commitment_id = (SELECT commitment_id FROM commitments LIMIT 1);	Se documenta si usa Seq Scan (la columna no tiene índice; solo existen los de llaves primarias y únicos)	Ejecutado el 08/11/2026 en el SQL Editor de Supabase. Seq Scan sobre treasury_movements (499 filas descartadas por el filtro); Execution Time: 0,221 ms. La columna no tiene índice.	Aprobado
PER-03	Consulta sobre más de 1 millón de registros	Tabla temporal de 1.000.000 filas (se descarta al terminar): agrupación y búsqueda por id sin y con índice.	Tiempo de respuesta menor a 2 segundos; el índice mejora la búsqueda	Ejecutado el 08/11/2026. Agrupación: 189,7 ms. Búsqueda por id sin índice: 248,3 ms; con índice: 0,8 ms (unas 320 veces más rápida).	Aprobado

Observaciones: Evaluación general: 3 de 3 casos aprobados. Problemas críticos identificados: ninguno; con 500 a 1000 filas las consultas responden en milisegundos y con un millón de filas en menos de 250 ms. Recomendaciones antes del despliegue: crear índices sobre las llaves foráneas (hoy solo existen los de llaves primarias y únicas), porque la búsqueda sin índice crece con el volumen.

Pruebas de Recuperación ante Fallos

ID	Caso de Prueba	Consulta SQL	Resultado Esperado	Resultado Obtenido	Estado (Aprobado / Fallo / Pendiente)
REC-01	Restauración de backup	En Supabase, en un bloque que se revierte al final: para projects, project_activities, commitments, treasury_movements y sales se crea una tabla de respaldo (CREATE TABLE AS SELECT), una copia de trabajo, se borran todas sus filas (falla) y se restauran desde el respaldo (INSERT ... SELECT); se comparan conteo y checksum md5 con la tabla real.	Los datos deben recuperarse sin pérdida	Ejecutado el 09/11/2026 en el SQL Editor de Supabase (Figura 14). Las cinco tablas quedaron con el mismo conteo tras la pérdida total y la restauración (500, 1000, 500, 500 y 200 filas; 0 filas tras la pérdida) y con checksum IGUAL. Luego se verificó que no quedó ninguna tabla auxiliar (29 tablas en public) y que los conteos de la base no cambiaron.	Aprobado
REC-02	Caída de la conexión en transacción activa	Sesión A: BEGIN; INSERT en banks; UPDATE de projects; pg_sleep(150) sin COMMIT. Sesión B: pg_terminate_backend sobre la sesión A en medio de la transacción.	La base mantiene la consistencia tras la recuperación	Ejecutado el 09/11/2026 en dos sesiones del SQL Editor de Supabase (Figuras 15 y 16). La sesión A estaba activa con la transacción abierta y fue terminada. Después: 0 sesiones A activas, banks sigue en 15 filas (la fila sin confirmar no existe), el UPDATE no se aplicó (progress = 85 y updated_at = 2026-11-07, sin cambio) y 0 compromisos con neto inconsistente. Es la caída de una conexión: el servidor gestionado de Supabase no se puede apagar.	Aprobado

Observaciones: Evaluación general: 2 de 2 casos aprobados. Problemas críticos identificados: ninguno; la restauración desde un respaldo no perdió datos y, al cortar la conexión en medio de una transacción, PostgreSQL descartó lo no confirmado y la base quedó consistente. Las pruebas se hicieron en la base de Supabase sin dejar cambios: REC-01 corre en un bloque que se revierte y REC-02 solo modifica filas dentro de una transacción que nunca se confirma. Recomendaciones antes del despliegue: configurar copias de seguridad periódicas en Supabase y repetir la restauración de prueba en un proyecto aparte.

Criterios de Éxito y Aprobación

- **Todos los casos críticos de integridad y de lógica en la base de datos (INT y TRG) deben aprobarse.**
- **Los hallazgos de seguridad (SEC-01 y SEC-02) quedan documentados y se resuelven con las recomendaciones del capítulo 8.**
- **Las consultas de reporte deben responder en menos de 2 segundos sobre el volumen de datos poblado.**

6. Producción

6.1 Desplegar el modelo físico en la nube (supabase)

https://supabase.com/

La base de datos está desplegada en Supabase (PostgreSQL 17.11 gestionado): proyecto base_de_datos, organización Pruebas, región us-east-1 (N. Virginia), plan Free. Pasos: (1) crear el proyecto en Supabase; (2) abrir el SQL Editor y ejecutar el script DDL de la sección 4.2 (tablas, funciones y triggers); (3) cargar los catálogos y los datos sintéticos desde los archivos CSV o Excel generados con IA (Anexo 1, E3 a E7); (4) verificar la carga contando las filas de cada tabla; y (5) conectar pgAdmin 4 con el pooler de sesión de Supabase (host aws-0-us-east-1.pooler.supabase.com, puerto 5432, base postgres, usuario postgres.<id del proyecto>, SSL requerido) para ejecutar consultas. Identificador del proyecto: sjrfuvfchxgpygvqyunx. URL del panel de Supabase: https://supabase.com/dashboard/project/sjrfuvfchxgpygvqyunx (requiere iniciar sesión). URL de la API del proyecto: https://sjrfuvfchxgpygvqyunx.supabase.co.

Las Figuras 4 a 6 muestran el proyecto desplegado en Supabase (estado Healthy, región East US, N. Virginia), una de sus tablas con registros y la conexión desde pgAdmin 4, donde el esquema public muestra las 29 tablas.

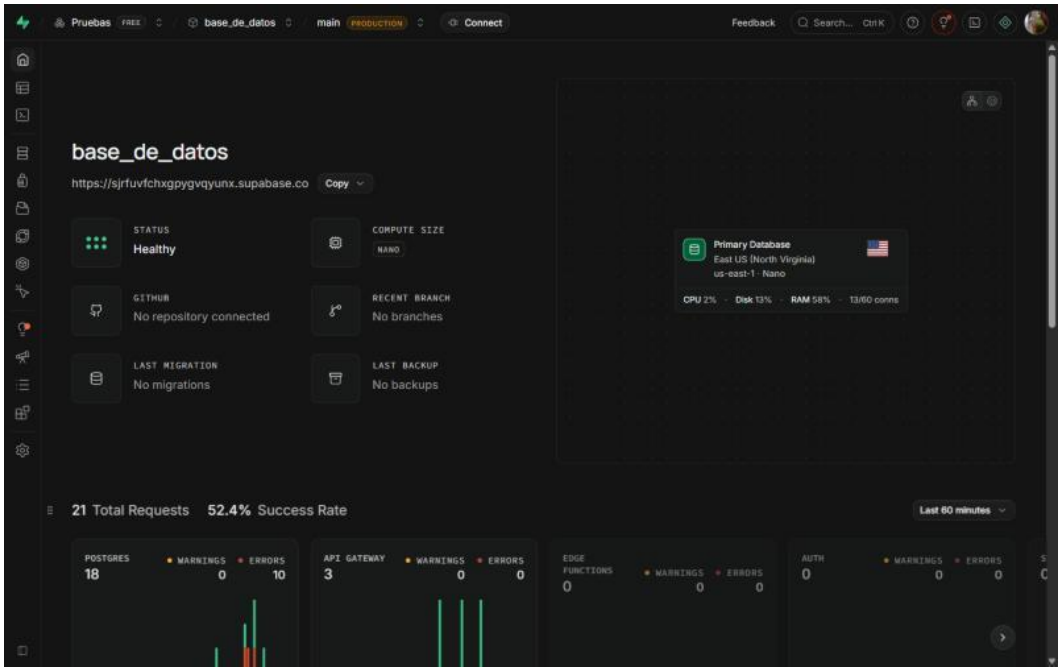


Figura 4. Proyecto base_de_datos en Supabase: estado Healthy, región us-east-1.

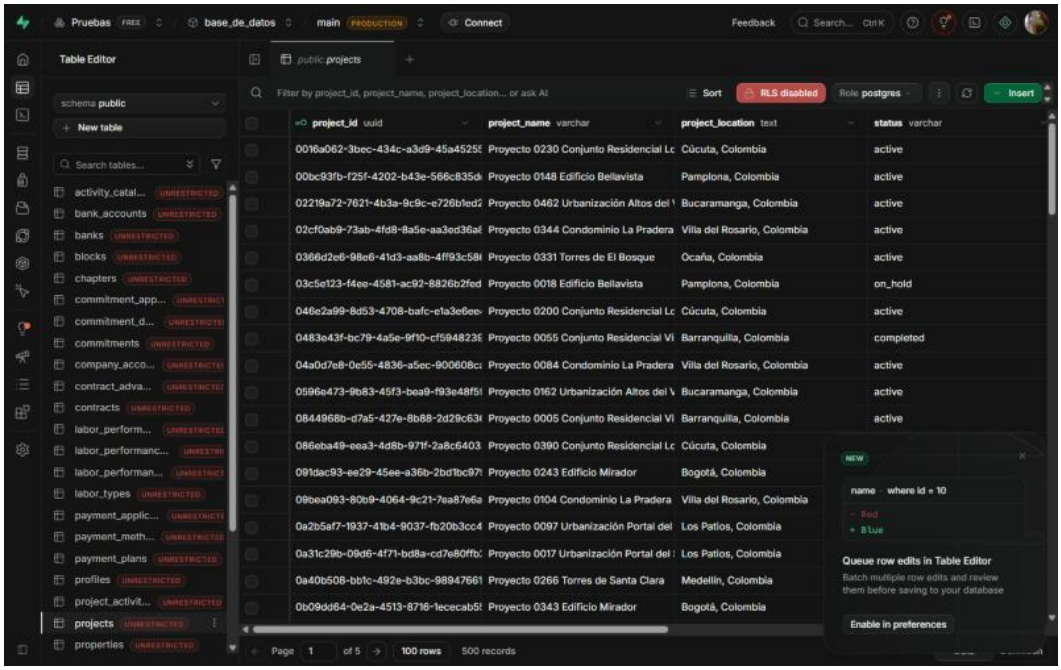


Figura 5. Table Editor de Supabase: tabla projects con 500 registros (el panel indica que RLS está desactivado).

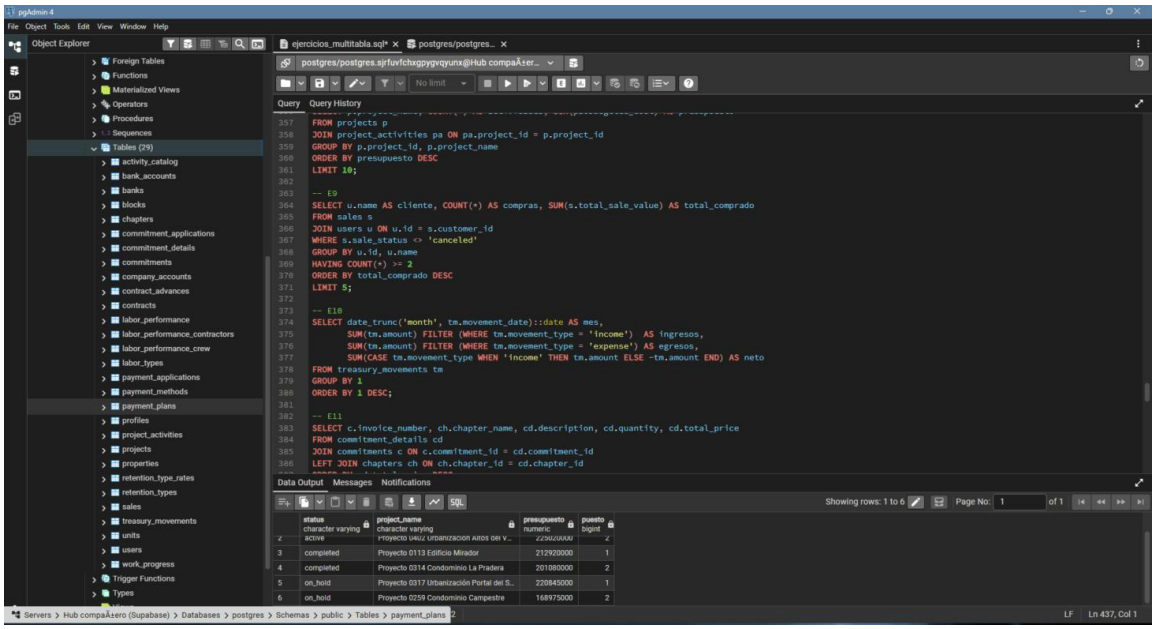
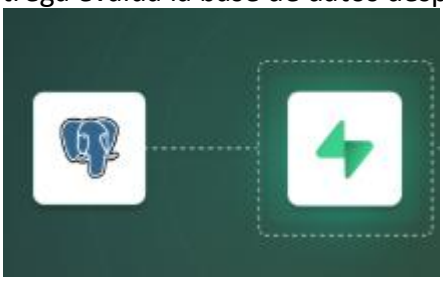


Figura 6. pgAdmin 4 conectado al servidor de la copia: el esquema public con sus 29 tablas.

6.2 Colocar en funcionamiento el sistema

La puesta en funcionamiento se verifica con consultas y operaciones de manipulación de datos sobre la base desplegada: se conecta pgAdmin 4 (o el SQL Editor de Supabase) al proyecto, se confirma el conteo de filas por tabla (500 proyectos, 1000 actividades de proyecto, 500

compromisos, 500 movimientos de tesorería y 200 ventas, entre otros) y se ejecutan las consultas con JOIN y funciones agregadas del plan UAT. Para conectar la aplicación web (Next.js 16) a esta base se configuran las variables NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY y SUPABASE_SERVICE_ROLE_KEY (esta última solo en el servidor, nunca expuesta al navegador). Evidencias del funcionamiento: las Figuras 7 a 13 muestran el conteo de filas, una consulta con JOIN y otra con funciones agregadas (en el SQL Editor de Supabase y en pgAdmin 4) y las operaciones INSERT, UPDATE y DELETE sobre el catálogo banks, que dejaron la tabla como estaba (15 filas). La aplicación web no se publica: esta entrega evalúa la base de datos desplegada en Supabase.



Las capturas siguientes se tomaron el 08/11/2026 (Figuras 7 a 13) y el 09/11/2026 (Figuras 14 a 16, pruebas de recuperación). Las consultas se ejecutaron en el SQL Editor de Supabase (Figuras 7, 8, 9 y 11 a 16) y en pgAdmin 4 (Figura 10).

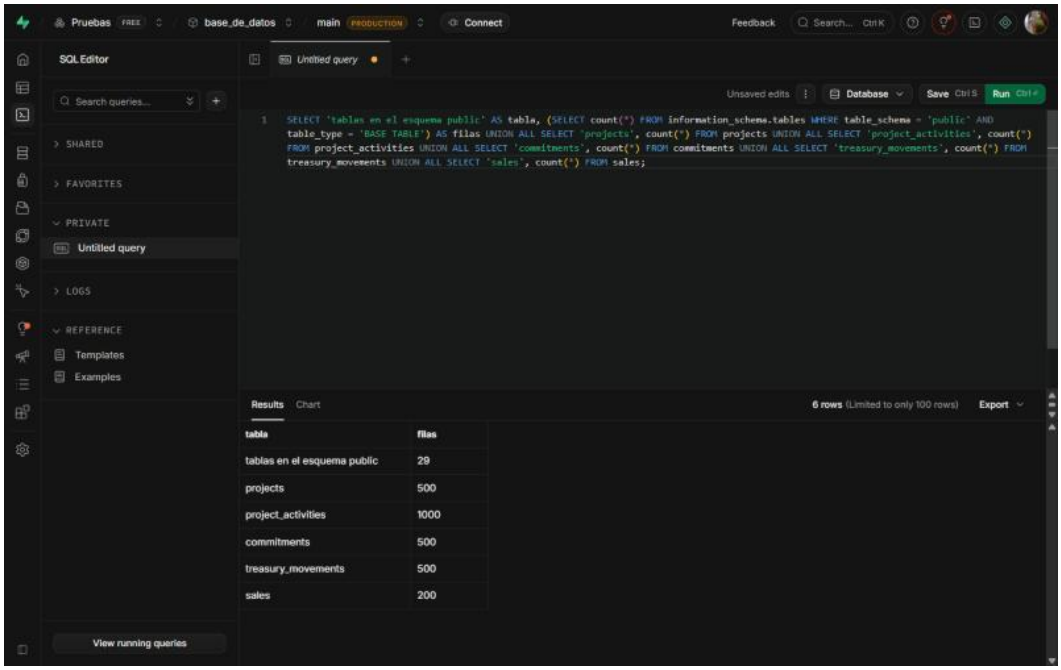


Figura 7. Conteo de las tablas del esquema public (29) y de las tablas principales: 500 projects, 1000 project_activities, 500 commitments, 500 treasury_movements y 200 sales.

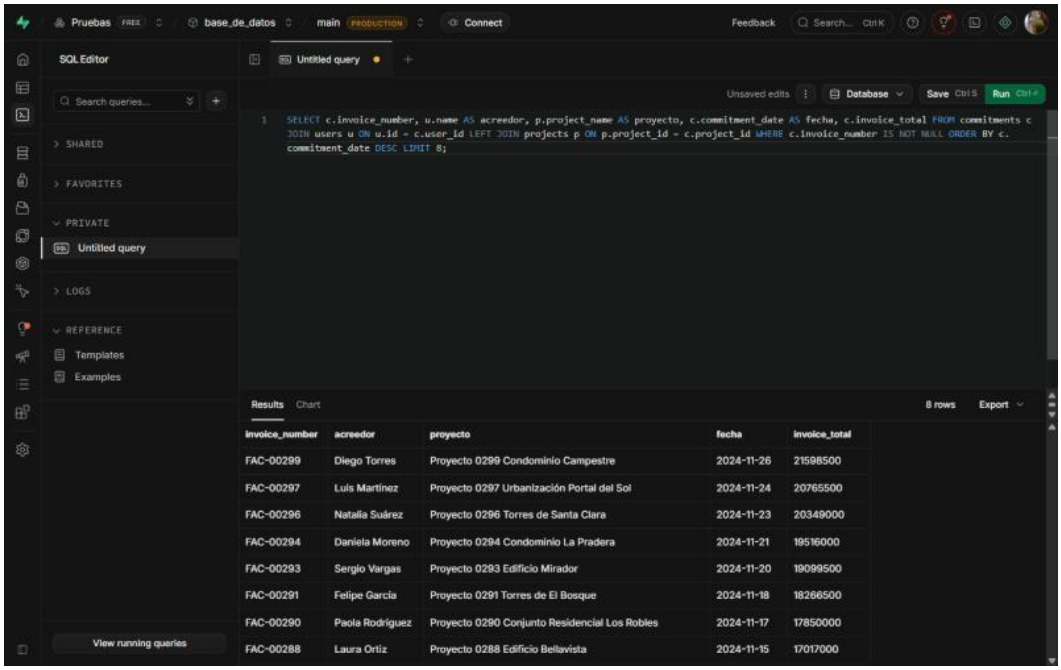


Figura 8. Consulta con JOIN de tres tablas (commitments, users y projects).

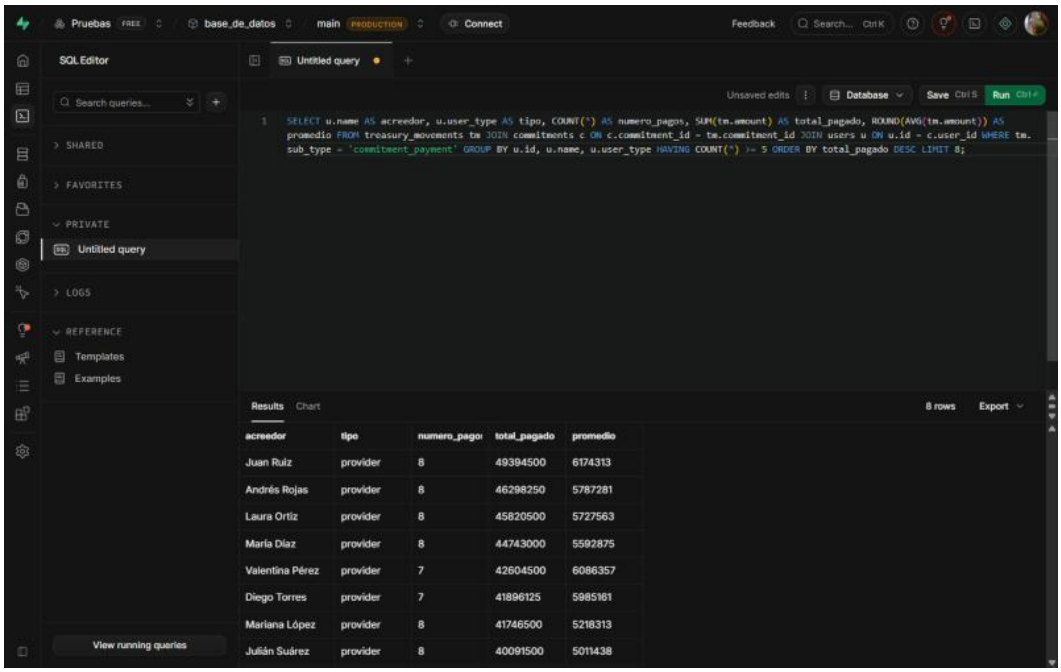


Figura 9. Consulta con funciones agregadas COUNT, SUM y AVG, GROUP BY y HAVING.

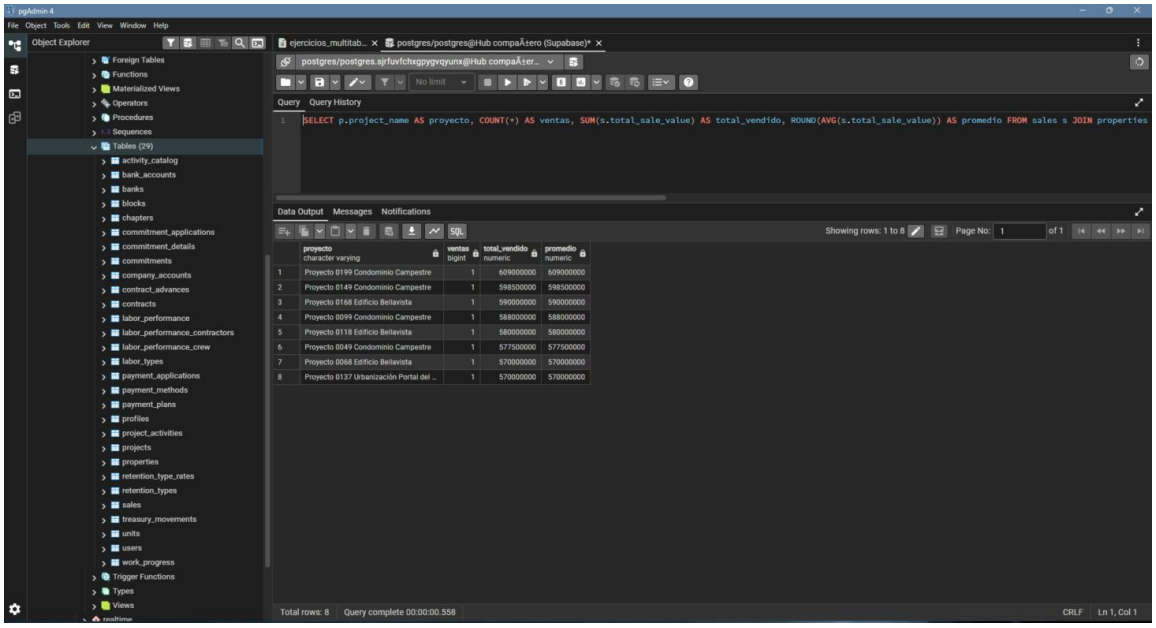


Figura 10. Consulta con JOIN y agregados ejecutada desde pgAdmin 4 (ventas por proyecto).

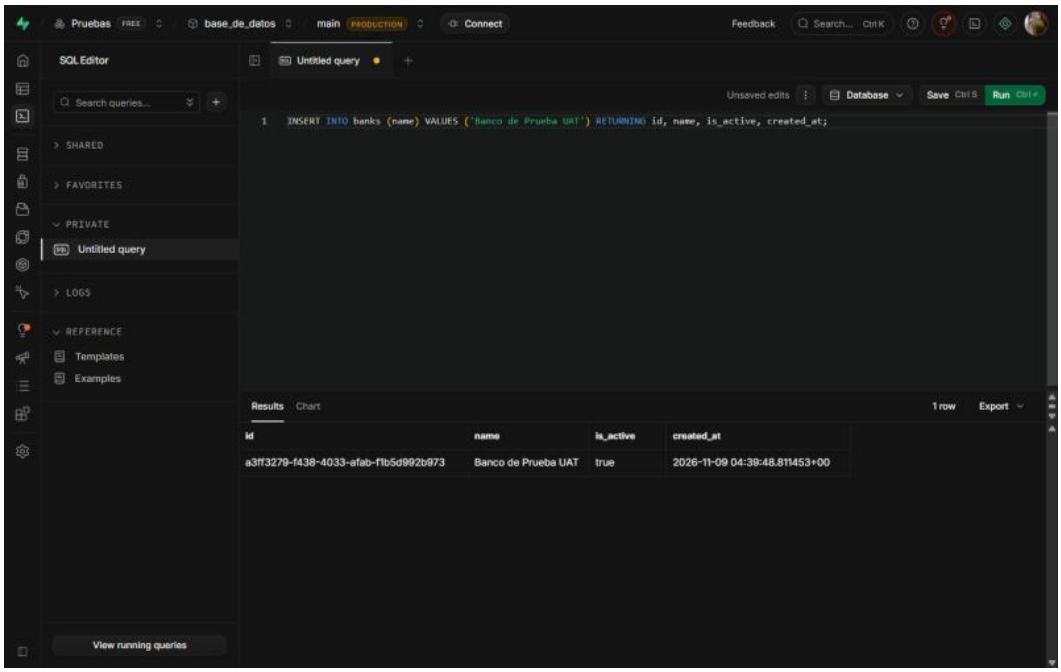


Figura 11. INSERT de una fila de prueba en el catálogo banks (manipulación de datos).

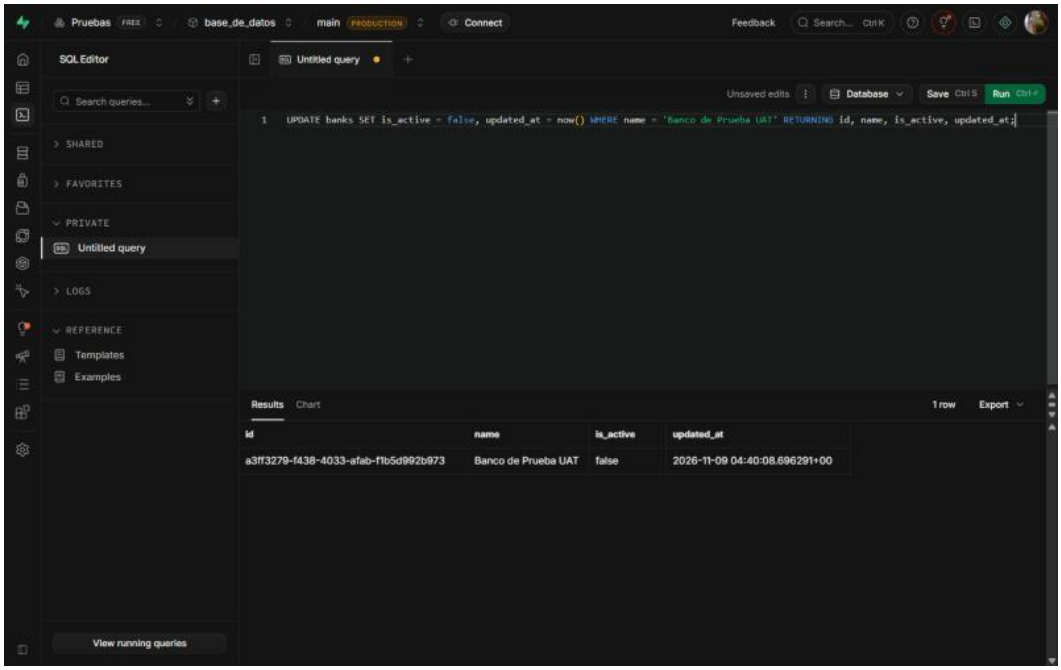


Figura 12. UPDATE de la fila de prueba (is_active = false).

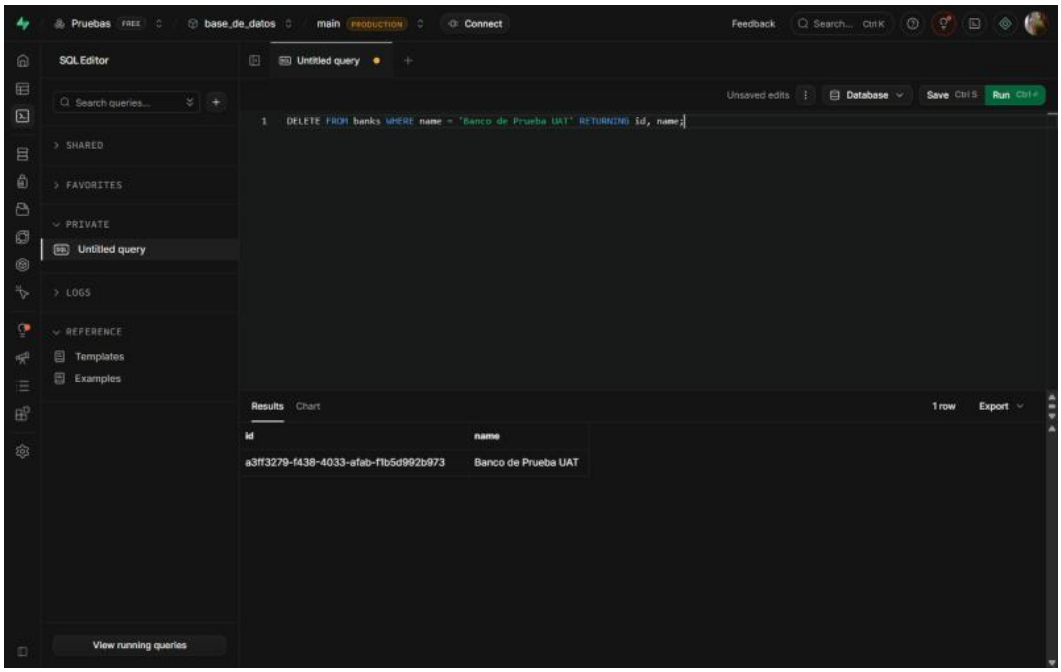


Figura 13. DELETE de la fila de prueba; después la tabla banks volvió a tener 15 filas.

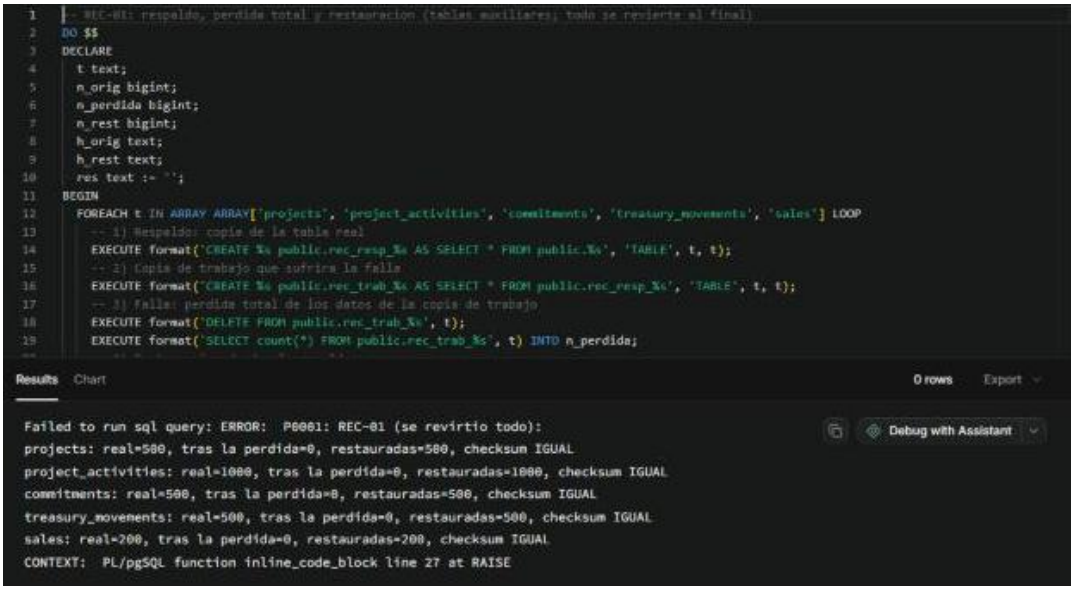


Figura 14. REC-01 en Supabase: respaldo, pérdida total y restauración de cinco tablas; los conteos y el checksum coinciden y el bloque se revierte.



Figura 15. REC-02 en Supabase: la sesión B corta la conexión de la sesión A, que tenía una transacción abierta (pg_terminate_backend).

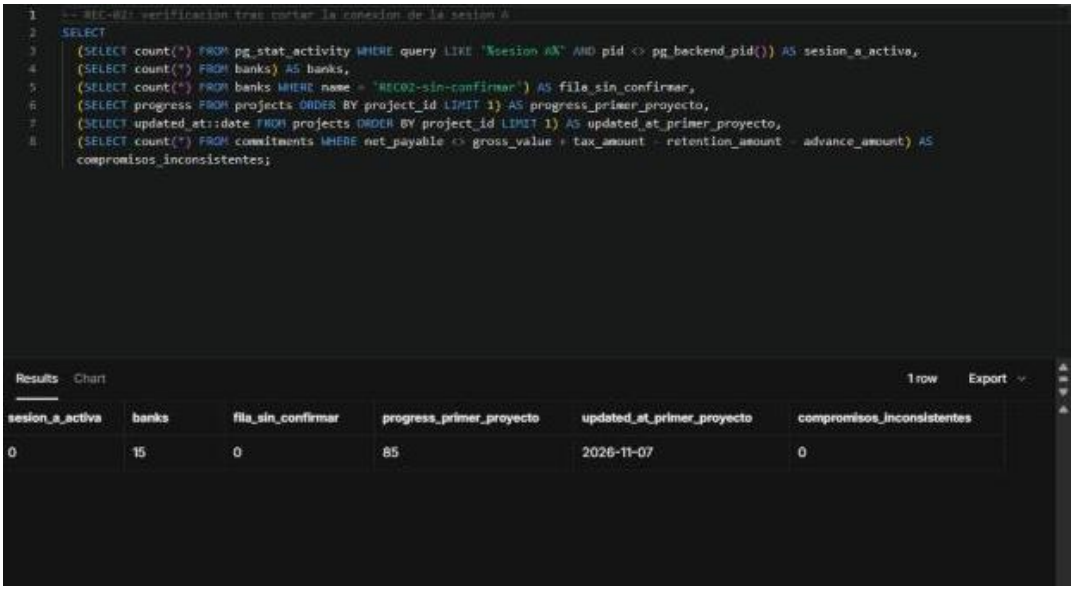


Figura 16. REC-02 en Supabase: tras el corte no queda la fila sin confirmar, el UPDATE no se aplicó y la base sigue consistente.

7. Conclusión

La base de datos diseñada e implementada para Hub TuCasaConstrucciones cubre el ciclo de la constructora en 29 tablas: terceros y acceso, proyectos e inmuebles, ventas con plan de pagos, compromisos con proveedores y contratistas, contratos y actas de avance, tesorería y control de obra. El modelo está normalizado hasta la tercera forma normal y protege la integridad con llaves primarias y foráneas (incluidas llaves compuestas), restricciones CHECK y UNIQUE, columnas calculadas y cinco triggers que automatizan reglas del negocio (estado del inmueble según la venta, aplicación FIFO de los pagos, bloque de vivienda y tipo de contratista). Se desplegó en Supabase (PostgreSQL 17) y se pobló con datos sintéticos generados con IA (500 proyectos, 1000 actividades de proyecto, 500 compromisos, 500 movimientos de tesorería y 200 ventas, entre otros) para practicar consultas con varias tablas. Se ejecutó el plan UAT: 26 de 28 casos aprobados y 2 hallazgos de seguridad (la base no activa RLS y el rol anónimo puede leer las tablas). Quedan como trabajo futuro la seguridad por filas (RLS) y las reglas señaladas en la trazabilidad (acta de entrega, observaciones comerciales y tipologías).

8. Recomendaciones

Repetir el plan UAT (archivo uat_copia.sql) antes de cada despliegue, registrar los resultados y probar una restauración de backup en un proyecto aparte.

Activar RLS en las 29 tablas con políticas por rol (superadmin, admin, accountant, foreman y assistant) y revocar al rol anon los permisos innecesarios: hoy ninguna tabla tiene RLS y el rol anon tiene SELECT sobre las tablas (hallazgos SEC-01 y SEC-02), por lo que los datos pueden quedar accesibles con la clave anónima de Supabase.

Agregar las restricciones que faltan: UNIQUE (document_type, document_number) en users; un índice único parcial sobre sales (property_id) donde sale_status <> 'canceled'; y una restricción de exclusión (extensión btree_gist) para impedir vigencias traslapadas en retention_type_rates.
Crear vistas para los saldos y estados (saldo de facturas, estado de cuotas y saldo de cuentas) en lugar de repetir las consultas.
Crear índices sobre las llaves foráneas más consultadas (por ejemplo treasury_movements.commitment_id y sale_id, commitments.user_id y project_id, sales.property_id), porque hoy solo existen los índices de llaves primarias y únicos.
Modelar las reglas pendientes: tipologías con precio base, acta de entrega material, observaciones comerciales por cliente y la regla de mora a 30 días.
Automatizar el cálculo de IVA y retenciones a partir de retention_type_rates; hoy se digita manualmente.
Ampliar los pagos de prueba: solo 150 de los 500 movimientos son pagos de compromisos y ninguna factura queda pagada por completo; al ampliar los datos conviene generar pagos que dejen facturas pagadas, parciales y pendientes.
Configurar copias de seguridad periódicas y conservar los documentos contables y legales por un mínimo de 10 años.
Mantener el script del esquema versionado y registrar cada cambio de esquema en una migración.

9. Glosario

- **Acta de entrega material: Documento oficial firmado por la constructora y el comprador en el cual se formaliza la entrega física del inmueble, dejando constancia del estado en que se recibe y cualquier observación o salvedad técnica.**

- **Crédito Hipotecario: Préstamo a mediano o largo plazo otorgado por una entidad financiera al comprador para cubrir el saldo restante del valor de la vivienda, utilizando el mismo inmueble como garantía de pago.**

- **Cuota de separación: Monto de dinero inicial que un cliente paga a la constructora para reservar una unidad de vivienda específica, bloqueando su disponibilidad comercial para otros interesados.**

- **Cuota inicial: Porcentaje del valor total del inmueble (usualmente entre el 20% y el 30%) que el comprador debe pagar con recursos propios, generalmente diferido en cuotas mensuales durante la etapa de construcción del proyecto.**

- **Escritura pública: Documento legal y notarial que formaliza el traspaso definitivo del derecho de dominio (propiedad) del inmueble por parte de TuCasa Construcciones hacia el comprador.**

- **Mora: Estado de incumplimiento financiero en el que entra un cliente cuando se retrasa en el pago de una o más cuotas establecidas en su plan de pagos (generalmente superando los 30 días de retraso).**

- **Obra gris habitable: Modalidad de entrega en la que el inmueble cuenta con toda su estructura, cerramientos y redes de servicios públicos 100% funcionales, pero se entrega sin los acabados estéticos finales (sin pisos instalados, pintura final o carpintería detallada), permitiendo al comprador personalizarla.**

- **Plan de pagos: Cronograma financiero pactado entre el comprador y la constructora al momento de firmar la promesa, donde se detallan las fechas, los montos exactos y la forma en que se pagará el inmueble.**

- **Promesa de compraventa: Contrato legal de carácter preparatorio mediante el cual la constructora y el comprador se obligan mutuamente a celebrar el negocio definitivo bajo unas condiciones específicas de precio, tiempo y entrega.**

- **Prospecto: Persona natural o jurídica que ha mostrado interés comercial en los proyectos de la constructora (cotizando o visitando la sala de ventas), pero que aún no ha firmado un contrato de separación.**

- **Tipología: Modelo estandarizado de diseño dentro de un proyecto inmobiliario (ejemplo: Casa Tipo A, Casa Tipo B) que determina el área construida, la distribución arquitectónica y el precio base del inmueble.**

- **Unidad de vivienda: Espacio habitacional individualizado e independiente (ya sea una casa o un apartamento) que hace parte integral de un proyecto constructivo, como una urbanización o conjunto multifamiliar.**

Compromiso: Obligación real de pago con un tercero (factura, cuenta de cobro o liquidación de contrato).
Acta de avance: Documento que certifica la ejecución de un contratista para efectos de pago y jurídicos.
Caja menor: Cuenta única de la empresa para gastos pequeños, alimentada con retiros de la cuenta bancaria.
Retención: Valor que se descuenta de una factura por concepto de impuestos (por ejemplo, ReteFuente o ReteICA).
RLS (Row Level Security): Mecanismo de PostgreSQL que limita las filas que cada rol puede ver o modificar.
ERD: Diagrama entidad-relación.
C4: Modelo para describir la arquitectura de software en niveles de contexto, contenedores, componentes y código.

Trigger: Función que la base de datos ejecuta automáticamente antes o después de insertar, actualizar o eliminar filas de una tabla.
Columna generada: Columna cuyo valor calcula la base de datos a partir de otras columnas de la misma fila (GENERATED ALWAYS AS ... STORED).
FIFO: Primero en entrar, primero en salir; aquí, los pagos de una venta se aplican primero a la cuota más antigua pendiente.
Datos sintéticos: Datos de prueba generados artificialmente (con IA) que no corresponden a personas ni transacciones reales.
Pooler de sesión: Servicio de Supabase que permite conectar clientes como pgAdmin a la base de datos por una dirección compatible con IPv4.

10. Bibliografía

Brown, S. (s. f.). The C4 model for visualising software architecture. https://c4model.com
Congreso de Colombia. (2011). Ley 1480 de 2011, Estatuto del Consumidor. Diario Oficial.
Elmasri, R. y Navathe, S. B. (2016). Fundamentals of database systems (7.ª ed.). Pearson.
PostgreSQL Global Development Group. (s. f.). PostgreSQL documentation. https://www.postgresql.org/docs/
Supabase. (s. f.). Supabase documentation. https://supabase.com/docs
Vercel. (s. f.). Next.js documentation. https://nextjs.org/docs

Anexo 1. Instrumento de recolección de evidencias

Este anexo reúne las evidencias del proceso de generación de datos sintéticos con IA para poblar la base de datos del proyecto de clase. El primer lote de datos (500 proyectos, 1000 actividades de proyecto, 500 compromisos, 500 movimientos de tesorería y 200 ventas, entre otros) fue generado con IA por un integrante del equipo, pero no se conservaron el prompt ni los archivos intermedios de ese lote. Por eso este anexo documenta con evidencia completa la carga complementaria del 09/11/2026, hecha por el equipo con IA para corregir los vacíos que detectó la revisión de calidad (E4).

Evidencia	Descripción	Contenido / estado
E1. Diseño relacional	Esquema de las tablas y sus restricciones.	Incluido: sección 3.1 (Tabla 1 y diccionario de datos de las 29 tablas) y script completo en la sección 4.2.
E2. Prompt inicial	Primera solicitud realizada a la IA. Prompt inicial y explicación de por qué se incluyeron esas instrucciones (indicar el LLM utilizado).	Incluido: LLM, prompt y justificación de la carga complementaria del 09/11/2026 (sección E2).
E3. Excel inicial	Archivo Excel original generado por la IA en el primer intento (se envía como archivo adjunto).	Incluido: archivo datos\datos_complementarios_2026-11-09.xlsx (carga complementaria); el primer lote no conservó su archivo.
E4. Registro de revisión	Errores encontrados y verificaciones realizadas del archivo Excel generado por la IA.	Incluido: 33 verificaciones SQL antes y después de la corrección (sección E4).
E5. Historial de interacción	Prompts adicionales y respuestas utilizadas para corregir los datos, con la identificación de errores y la justificación de las modificaciones solicitadas.	Incluido: historial de interacción de la carga complementaria (sección E5).
E6. Excel definitivo	Archivo validado antes de la importación (se envía como archivo adjunto).	Incluido: es el mismo archivo del E3 (el primer intento fue el definitivo).
E7. Base de datos poblada	Tablas con los registros importados en PostgreSQL: capturas o scripts de importación (COPY) y registros cargados.	Incluido: filas agregadas, método de carga, capturas de la prueba y de la confirmación, y filas por tabla (sección E7).
E8. Validación SQL	Consultas utilizadas para comprobar la integridad de los datos y explicación de los resultados.	Incluido: consultas de validación con sus resultados (sección E8) y plan UAT (sección 5).
E9. Reflexión individual	Explicación de las decisiones y aprendizajes obtenidos.	Incluidas las reflexiones de los tres integrantes (sección E9).

E2. Prompt inicial y LLM utilizado (carga complementaria del 09/11/2026)

LLM utilizado: Claude, de Anthropic (modelo Claude Sonnet 5.5, claude-sonnet-5-5), usado desde Claude Code en el computador del integrante Santiago Castilla.

Prompt (texto del integrante): "De los 3 hallazgos reales que encontraste de los datos de las tablas, arregla la base de datos para que quede mejor poblada para hacer las consutlas mas adecuadas para el profesor si las pide ya que devis que hay muchas cosas vacias".

Por qué se incluyeron esas instrucciones: se pidió corregir solo los 3 hallazgos de la revisión (470 compromisos sin ítems, 250 proyectos sin inmuebles y 20 proveedores y contratistas sin cuenta bancaria) para que las consultas de varias tablas que pida el profesor no devuelvan resultados vacíos.

Condiciones con las que trabajó la IA, y por qué: (1) respetar el esquema real de 29 tablas, para que las llaves foráneas y las restricciones CHECK no fallaran; (2) mirar primero los datos existentes (consulta de patrones), para que los nuevos tuvieran el mismo formato (códigos P-####-101, precios de 3 a 5,25 millones por m², cuentas de 10 dígitos, un bloque de viviendas por proyecto); (3) no cambiar las cantidades pedidas por el profesor (500, 1000, 500, 500 y 200); (4) generar datos deterministas con hashtext, para que la carga sea reproducible; (5) probar primero en modo prueba (v_dry = true), que valida y revierte todo, y solo después confirmar.

E3 y E6. Excel inicial y Excel definitivo

En esta carga el primer intento fue el definitivo: la prueba en modo v_dry pasó las 7 validaciones sin necesidad de corregir los datos, así que el archivo inicial y el definitivo son el mismo: datos\datos_complementarios_2026-11-09.xlsx, con cuatro hojas (commitment_details: 938 filas, commitment_applications: 150, properties: 250 y bank_accounts: 20). Los datos se generaron directamente en PostgreSQL con INSERT ... SELECT (script completar_datos.sql) y el archivo Excel es la exportación, hecha desde pgAdmin 4, de las filas cargadas.

E4. Registro de revisión: calidad de los datos, antes y después de la corrección (33 verificaciones con SQL)

Revisión de coherencia y de formato de los datos importados. El 09/11/2026 la primera pasada encontró 3 hallazgos (C21, F09 y F11), 28 verificaciones sin inconsistencias y 2 informativas. Los hallazgos se corrigieron con la carga complementaria (E7) y la misma revisión se repitió después: las 33 verificaciones quedan sin hallazgos. La revisión se hizo sobre la base poblada; los errores que el equipo encontró en el Excel antes de importarlo no se conservaron.

Verificación	Antes	Después	Lectura
C01 proyectos completed con progress menor a 100	0	0	Sin inconsistencias.
C02 proyectos active con progress igual a 100	0	0	Sin inconsistencias.
C03 nombres de terceros repetidos	0	0	Sin inconsistencias.
C04 correos o teléfonos repetidos	0	0	Sin inconsistencias.
C05 ventas a terceros que no son clientes	0	0	Sin inconsistencias.
C06 compromisos con acreedor de tipo client	0	0	Sin inconsistencias.
C07 pagos con proyecto distinto al del compromiso	0	0	Sin inconsistencias.
C08 pagos de venta mayores al valor de la venta	0	0	Sin inconsistencias.
C09 ventas anteriores al inicio del proyecto	0	0	Sin inconsistencias.
C10 compromisos anteriores al inicio del proyecto	0	0	Sin inconsistencias.
C11 inmuebles sold sin venta vigente	0	0	Sin inconsistencias.
C12 precio por m² mínimo y máximo	3.000.000 y 5.250.000	3.000.000 y 5.250.000	Rango razonable para vivienda; los inmuebles nuevos usan los mismos precios por m².
C13 actividades con fecha fin anterior al inicio	0	0	Sin inconsistencias.
C14 cuotas con vencimiento anterior a la venta	0	0	Sin inconsistencias.
C15 movimientos con tipo y subtipo incoherentes	0	0	Sin inconsistencias.
C16 retención o IVA mayores al valor bruto	0	0	Sin inconsistencias.
C17 contratos con actas por encima del valor estimado	0	0	Sin inconsistencias.
C18 cuota inicial mayor al valor o separación mayor a la cuota inicial	0	0	Sin inconsistencias.
C19 movimientos o compromisos con fecha futura	0	0	Sin inconsistencias.
C20 compromisos sin proyecto	0	0	Sin inconsistencias.
C21 compromisos sin ítems	470	0	Corregido: los 470 compromisos sin ítems recibieron 938 ítems (ahora 1 a 3 por compromiso).

Verificación	Antes	Después	Lectura
F01 correos con formato inválido	0	0	Sin inconsistencias.
F02 teléfonos que no tienen 10 dígitos	0	0	Sin inconsistencias.
F03 documentos de tercero no numéricos	0	0	Sin inconsistencias.
F04 facturas con número distinto al formato FAC-00000	0	0	Sin inconsistencias.
F05 números de factura repetidos	0	0	Sin inconsistencias.
F06 números de cuenta bancaria repetidos	0	0	Sin inconsistencias.
F07 códigos de inmueble repetidos	0	0	Sin inconsistencias.
F08 proyectos sin bloques	0	0	Sin inconsistencias.
F09 proyectos sin inmuebles	250	0	Corregido: los 250 proyectos sin inmuebles recibieron uno cada uno (ahora hay 500 inmuebles).
F10 proyectos sin actividades presupuestadas	0	0	Sin inconsistencias.
F11 proveedores o contratistas sin cuenta bancaria	20	0	Corregido: los 20 proveedores y contratistas sin cuenta recibieron su cuenta principal (ahora 40 cuentas).
F12 terceros por tipo	client=20, contractor=20, employee=15, provider=20	client=20, contractor=20, employee=15, provider=20	Distribución de los 75 terceros.

Hallazgo 1 (corregido): 470 de los 500 compromisos no tenían ítems y commitment_applications estaba vacía. Ahora todos los compromisos tienen entre 1 y 3 ítems cuya suma es igual al valor bruto, y los 150 pagos de compromisos están aplicados a su compromiso.

Hallazgo 2 (corregido): 250 de los 500 proyectos no tenían inmuebles. Ahora todos tienen un inmueble en su bloque de viviendas.

Hallazgo 3 (corregido): 20 de los 40 proveedores y contratistas no tenían cuenta bancaria. Ahora todos tienen una cuenta principal.

Verificaciones adicionales después de la carga: la suma de los ítems es igual al valor bruto en los 500 compromisos (0 diferencias); ningún compromiso queda pagado por encima de su neto a pagar (0); los conteos pedidos por el profesor no cambiaron (500, 1000, 500, 500 y 200).

E5. Historial de interacción con la IA (carga complementaria)

N.º	Prompt del integrante	Qué hizo la IA	Resultado y justificación
1	"hace la parte 3" (pedido de revisar y completar la evidencia de los datos sintéticos).	Buscó en el computador los Excel y prompts del proyecto (no había) y, en su lugar, ejecutó una revisión crítica de la calidad de los datos con 33 verificaciones SQL.	28 sin inconsistencias, 2 informativas y 3 hallazgos (470 compromisos sin ítems, 250 proyectos sin inmuebles, 20 proveedores y contratistas sin cuenta). Justificación: sin los archivos originales, la forma honesta de evaluar la información era revisar los datos ya cargados.
2	"De los 3 hallazgos reales ... arregla la base de datos para que quede mejor poblada ..." (texto completo en E2).	Inspeccionó los patrones de los datos existentes, escribió completar_datos.sql, lo ejecutó en modo prueba (v_dry = true, que revierte todo) y, al pasar las validaciones, lo confirmó (v_dry = false); luego repitió las 33 verificaciones.	938 ítems, 150 aplicaciones de pago, 250 inmuebles y 20 cuentas bancarias; los 3 hallazgos quedaron en 0 y los conteos del profesor no cambiaron. Justificación: la prueba previa evita dejar datos a medias si una validación falla.

N.º	Prompt del integrante	Qué hizo la IA	Resultado y justificación
3	"mi compañero no tiene nada, hagamos todos nosotros" (el integrante que generó el primer lote no conservó el prompt ni los Excel).	Documentó este anexo como evidencia de la carga complementaria, sin atribuir a esta carga el proceso del primer lote, y exportó las filas cargadas a Excel.	Anexo 1 completo para la carga del 09/11/2026 y archivo datos_complementarios_2026-11-09.xlsx. Justificación: no inventar un historial que no existe.

E7. Base de datos poblada: carga complementaria del 09/11/2026

Filas agregadas por la carga complementaria (la prueba en modo v_dry y la confirmación se muestran en las Figuras 17 y 18):

Tabla	Antes	Después	Agregadas
commitment_details	30	968	938
commitment_applications	0	150	150
properties	250	500	250
bank_accounts	20	40	20

Método de importación: el script completar_datos.sql (sección 4.2) se ejecutó en pgAdmin 4. Es un bloque DO con INSERT ... SELECT deterministas que valida 7 condiciones (compromisos sin ítems, suma de ítems, pagos por encima del neto, proyectos sin inmuebles, códigos de inmueble repetidos, cuentas faltantes y cuentas repetidas) y, con v_dry = true, revierte todo al terminar. Primero se ejecutó con v_dry = true (PRUEBA OK: 938 ítems, 150 aplicaciones, 250 inmuebles y 20 cuentas) y después con v_dry = false (CARGA CONFIRMADA con las mismas cantidades).

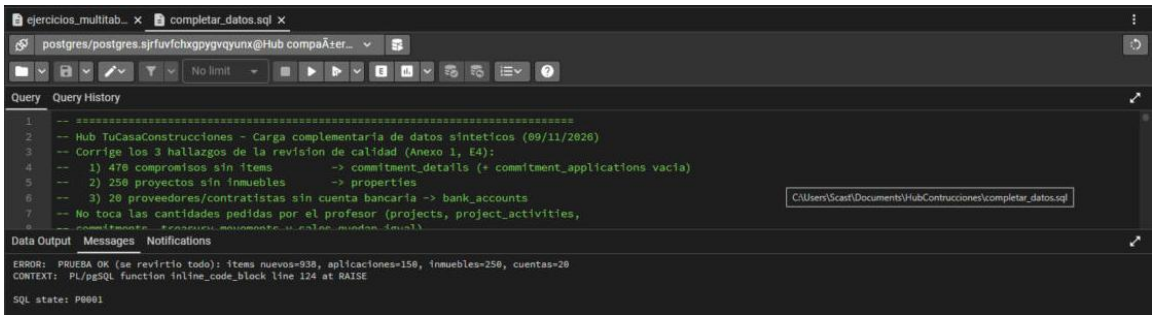


Figura 17. completar_datos.sql en modo prueba (v_dry = true): PRUEBA OK, se revirtió todo.

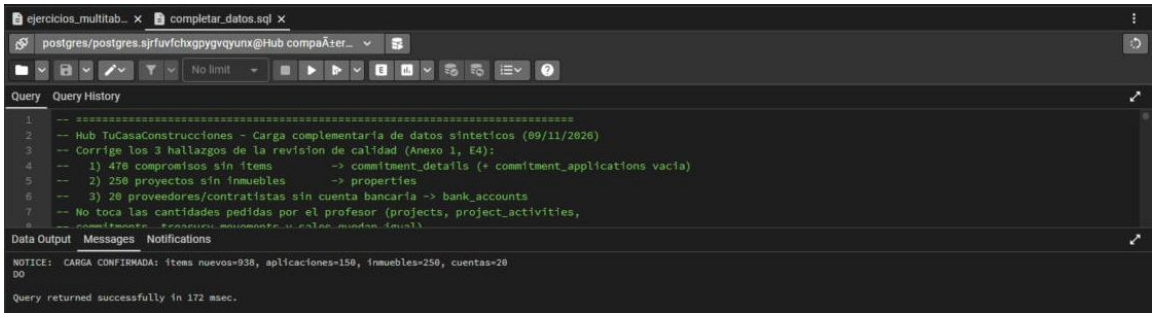


Figura 18. completar_datos.sql con v_dry = false: CARGA CONFIRMADA (938 ítems, 150 aplicaciones, 250 inmuebles y 20 cuentas).

E7. Cantidad de filas por tabla después de la carga (verificada con count(*) en la base desplegada el 09/11/2026)

Tabla	Filas
banks	15
payment_methods	15
chapters	15
units	15
retention_types	15
retention_type_rates	15
labor_types	15
profiles	15
users	75
bank_accounts	40
projects	500
blocks	1000
activity_catalog	500
properties	500
project_activities	1000
work_progress	40
labor_performance	20
labor_performance_contractors	20
labor_performance_crew	20
contracts	20
contract_advances	15
commitments	500
commitment_details	968
sales	200
payment_plans	600
company_accounts	15
treasury_movements	500
commitment_applications	150
payment_applications	250

E8. Validación SQL: consultas para comprobar la integridad de los datos

Consulta	Qué comprueba	Resultado
SELECT count(*) FROM <tabla>;	Que se cargaron las cantidades pedidas por el profesor: 500 projects, 1000 project_activities, 500 commitments, 500 treasury_movements y 200 sales.	Ejecutada: coincide con lo pedido (ver E7).

Consulta	Qué comprueba	Resultado
SELECT count(*) FROM commitments WHERE net_payable <> gross_value + tax_amount - retention_amount - advance_amount;	Que las columnas calculadas son consistentes.	Ejecutada el 08/11/2026: 0 compromisos con diferencia.
SELECT document_type, document_number, count(*) FROM users GROUP BY 1, 2 HAVING count(*) > 1;	Que no hay terceros con el mismo documento.	Ejecutada el 08/11/2026: 0 documentos duplicados.
SELECT sale_id FROM sales s JOIN properties p ON p.property_id = s.property_id WHERE s.sale_status <> 'canceled' AND p.status <> 'sold';	Que las ventas vigentes coinciden con el estado del inmueble.	Ejecutada el 08/11/2026: 0 filas (sin inconsistencias).
SELECT project_id FROM projects WHERE end_date < start_date;	Que las fechas de los proyectos son coherentes.	Ejecutada el 08/11/2026: 0 filas (sin inconsistencias).
SELECT c.commitment_id FROM commitments c JOIN treasury_movements tm ON tm.commitment_id = c.commitment_id AND tm.sub_type = 'commitment_payment' GROUP BY c.commitment_id, c.net_payable HAVING SUM(tm.amount) > c.net_payable;	Que no se pagó más de lo debido en una factura.	Ejecutada el 08/11/2026: 0 filas (sin inconsistencias).
SELECT property_id FROM sales WHERE sale_status <> 'canceled' GROUP BY property_id HAVING count(*) > 1;	Que ningún inmueble tiene dos ventas vigentes.	Ejecutada el 08/11/2026: 0 filas (sin inconsistencias).
SELECT s.sale_id FROM sales s JOIN payment_plans pp ON pp.sale_id = s.sale_id GROUP BY s.sale_id, s.total_sale_value HAVING SUM(pp.expected_amount) <> s.total_sale_value;	Que la suma de las cuotas de cada venta es igual a su valor total.	Ejecutada el 08/11/2026: 0 filas (sin inconsistencias).
SELECT pp.plan_id FROM payment_plans pp JOIN payment_applications pa ON pa.plan_id = pp.plan_id GROUP BY pp.plan_id, pp.expected_amount HAVING SUM(pa.applied_amount) > pp.expected_amount;	Que no se aplicó a una cuota más de su valor.	Ejecutada el 08/11/2026: 0 filas (sin inconsistencias).
SELECT d.commitment_id FROM commitment_details d JOIN commitments c ON c.commitment_id = d.commitment_id GROUP BY d.commitment_id, c.gross_value HAVING sum(d.total_price) <> c.gross_value;	Que la suma de los ítems es igual al valor bruto de cada compromiso (después de la carga).	Ejecutada el 09/11/2026: 0 filas (sin inconsistencias).
SELECT ca.commitment_id FROM commitment_applications ca JOIN commitments c ON c.commitment_id = ca.commitment_id GROUP BY ca.commitment_id, c.net_payable HAVING sum(ca.applied_amount) > c.net_payable;	Que ningún compromiso queda pagado por encima de su neto a pagar.	Ejecutada el 09/11/2026: 0 filas (sin inconsistencias).

E9. Reflexión individual

Santiago Castilla Coronel. Decidí que el documento describiera la base tal como está desplegada en Supabase (29 tablas, 5 triggers, sin vistas ni RLS) y que las pruebas se ejecutaran de verdad. Al revisar los datos con consultas encontré tres vacíos: 470 compromisos sin ítems, 250 proyectos sin inmuebles y 20 proveedores sin cuenta bancaria. Los corregí con un script determinista, probado antes en modo prueba que revierte todo, y comprobé que las cantidades del profesor no cambiaran. Aprendí que los datos de la IA tienen huecos que solo se ven validándolos con SQL; y del modelo relacional, las llaves compuestas, las columnas calculadas y por qué el trigger FIFO duplica pagos si se carga a ciegas. Queda sin resolver la seguridad por filas (RLS).

Cristian Danilo Areniz Coronel. En este trabajo me encargué de revisar que los datos de la base de Hub TuCasaConstrucciones fueran coherentes con el modelo. Una decisión que tomé fue pedir cantidades exactas, las mismas que pidió el profesor (500 proyectos, 1000 actividades, 500 compromisos, 500 movimientos y 200 ventas), para poder verificar cada tabla con un COUNT y no confiar solo en lo que la IA decía haber generado. También insistí en que respetara el orden de las llaves foráneas, porque si un hijo se inserta antes que su padre, la base lo

rechaza. Entendí que algo que se inserta sin error no significa que esté completo. Si lo repitiera, guardaría cada prompt desde el primer lote, porque el primero no conservó el suyo y no se puede reproducir. Además, solo 150 de los 500 movimientos son pagos de compromisos, así que ninguna factura queda pagada por completo.

Luis Felipe Carrascal Ortiz. Trabajé sobre todo en las retenciones de las facturas. Desde la primera reunión con la empresa nos contaron que a las facturas les sacaban a mano cada retención, así que me di la tarea de crear una tabla nueva y asociarla con la de facturas. Se decidió que las retenciones fueran una tabla aparte y no un atributo de commitments porque las tarifas se actualizan con el tiempo: si el porcentaje fuera un número suelto en commitments, al cambiarlo no se sabría con qué porcentaje se calcularon las facturas viejas. Al crear la tabla le pedí a la IA que respetara la estructura de las llaves foráneas, que es esencial para que funcionen las consultas. La IA generó un error en una llave foránea entre retention_type_rates y commitments; lo descubrí al correr el script en PostgreSQL, que lanzó el error, y tuve que corregirlo yo en la tabla porque la llave estaba mal puesta. Lo diferente que haría es revisar el script que me da la IA antes de ejecutarlo. Con este proceso aprendí cómo interactúan las llaves primarias y foráneas y por qué cada restricción es importante para el funcionamiento de la base. La única limitación que observo por ahora es que el campo retention_amount todavía debe llenarse a mano.